CertOpt: Certifying SQL Rewrites via Counterexample-Guided Synthesis

Subhas Kumar Ghosh
Westpac Banking Corporation
Sydney, NSW
subhas.ghosh@westpac.com.au

Vijay Monic Vittamsetti
Westpac Banking Corporation
Sydney, NSW
vijaymonic.vittamsetti@westpac.com.au

ABSTRACT

SQL query optimizers routinely transform queries without proving that rewrites preserve semantics, and bugs in these transformations have caused hundreds of silent wrong-result errors. We present CertOpt, a *proof-carrying* query optimization framework built on Counterexample-Guided Inductive Synthesis (CEGIS). CertOpt generates rewrite candidates via algebraic rules and LLM-assisted synthesis, verifies each candidate's equivalence under bounded bag semantics through SMT-based witness synthesis in Z3, and selects the cheapest certified rewrite. Accepted rewrites carry replayable equivalence certificates; rejected rewrites are accompanied by concrete distinguishing witness databases. Compositional per-join decomposition scales verification to queries with 6–16 tables. We evaluate on 24,455 VeriEQL equivalence pairs (zero false equivalence claims within the verifier's certified scope, up to 582× speedup over the state of the art), 30 JOB-Complex queries (27/30 certified), and 24,495 SQLStorm pairs where CertOpt catches 131 semantically incorrect LLM rewrites (3.1% rejection rate) with validated witnesses.

PVLDB Reference Format:
Subhas Kumar Ghosh and Vijay Monic Vittamsetti. CertOpt: Certifying SQL Rewrites via
Counterexample-Guided Synthesis. PVLDB, 19(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Query optimization is one of the oldest and most consequential components of a relational database system. A modern optimizer may apply dozens of algebraic rewrite rules—predicate pushdown, join reordering, subquery decorrelation, aggregate splitting—before selecting a physical plan. Each transformation is assumed to preserve the semantics of the original query, yet this assumption is rarely verified at the level of individual transformations. Production optimizers encode rewrite correctness as informal invariants maintained by human developers, and the consequences of violations are severe: silent wrong-result bugs that return incorrect data without raising any error. The SQLancer project [34] demonstrated the scale of this problem by discovering hundreds of logic bugs across more than twenty production-grade database systems, including SQLite, PostgreSQL, MySQL, CockroachDB, TiDB, and DuckDB.

Proceedings of the VLDB Endowment, Vol. 19, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

The rise of large language models (LLMs) as sources of SQL rewrites exacerbates the risk: recent work shows that 11.3–14.2% of text-to-SQL queries that pass execution-based evaluation are actually semantically incorrect [22], meaning that a substantial fraction of LLM-generated SQL is plausible-looking but semantically wrong. No existing query optimizer or database system provides a mechanism to formally certify that a proposed rewrite is equivalent to the original query, nor to explain *why* an incorrect rewrite is wrong by producing a concrete counterexample.

Two lines of work approach the problem from complementary directions but leave a critical gap. *SQL equivalence verifiers* (Cosette [9], VeriEQL [19], Polygon [45]) can decide bounded equivalence for query pairs, but operate as offline tools—none is integrated into an optimization loop. *Learned query optimizers* (Neo [29], Bao [28], Balsa [43], Lero [48]) improve plan selection but provide no semantic guarantee that rewritten SQL preserves the original result. Closing this gap is not straightforward: existing verifiers are too slow for per-candidate invocation (e.g., VeriEQL averages 33–73 s per pair when checking databases with up to 5 rows per table [19]), their encodings do not scale to the wide joins (6–16 tables) common in real workloads [31], and they lack a mechanism to reuse counterexamples across structurally related candidates. We observe that the *Counterexample-Guided Inductive Synthesis* (CEGIS) paradigm [36] maps naturally onto this problem: the *specification* is the original query, the *candidates* are proposed rewrites, and the *verifier* is a bounded equivalence checker backed by Z3 [13]. When verification fails, the solver returns a concrete counterexample that can prune entire families of structurally related candidates.

We present CertOpt, a *proof-carrying* query optimization framework that closes the generate–verify–certify loop. Our contributions are:

(1) **CEGIS-based proof-carrying optimization loop** (§3–4): a generate–verify–certify pipeline that produces rewrite candidates via algebraic rules and LLM synthesis, verifies each under bounded bag semantics, and selects the cheapest certified rewrite.
(2) **Faithful Z3 encoding of SQL bag semantics** (§5): three-valued NULL logic, inner/outer joins, grouped aggregation with `COUNT DISTINCT`, window functions, `ORDER BY` with `LIMIT`, and integrity constraints, with a conservative under-approximation for unmodeled constructs (Theorem 7.2).
(3) **Compositional verification for wide queries** (§6): per-join decomposition with a formal composition theorem (Theorem 6.1) enabling verification of queries with 6–16 tables—from 0/30 to 27/30 verified JOB-Complex queries.
(4) **Three-axis evaluation** (§8): verifier accuracy on 24,455 pairs (zero false equivalence claims, 241 confirmed VeriEQL

false equivalence proofs, up to 582× speedup), rewrite validation on 30 JOB-Complex queries, and LLM guardrail on SQLStorm workloads.

Motivating example. Consider the following Calcite optimizer rule that rewrites an `IN` subquery to an equivalent `JOIN`:

```
-- Original (Q):
SELECT e.* FROM emp e
WHERE e.deptno IN (SELECT d.deptno FROM dept d)

-- Rewrite (Q'):
SELECT e.* FROM emp e
JOIN (SELECT deptno FROM dept GROUP BY deptno) t
  ON e.deptno = t.deptno
```

Given the schema `EMP(empno PK, deptno FK→dept, ...)` and `DEPT(deptno PK, ...)`, CertOpt verifies this rewrite in 3 ms at k=3: the `GROUP BY` in the subquery deduplicates the join, preserving bag multiplicities. The certificate proves that Q and Q' produce identical results on all database instances with ≤ 3 rows per table.

Now consider a subtly incorrect variant proposed by an LLM that omits the `GROUP BY`:

```
-- Buggy rewrite (Q''):
SELECT e.* FROM emp e
JOIN dept d ON e.deptno = d.deptno
```

CertOpt immediately produces a distinguishing witness: `dept` = $\{(10), (10)\}$, `emp` = $\{(1, 10)\}$. The original returns one row; the join returns two (one per matching `dept` row). The concrete witness explains *why* the rewrite fails: duplicate department keys inflate result multiplicities. Without the `GROUP BY`, the join is not multiplicity-safe.

2 BACKGROUND

2.1 SQL Bag Semantics and Integrity Constraints

We formalize the fragment of SQL that CertOpt reasons about.

Definition 2.1 (Schema and Database Instance). *A* schema $\mathcal{S} = (R_1, \ldots, R_m)$ *is a finite sequence of relation schemas, where each R_i has a name and a tuple of typed column names. A* database instance D *conforming to $\mathcal{S}$ assigns to each R_i a finite* bag *(multiset) of tuples whose values are drawn from the corresponding column types, augmented with the distinguished value* NULL.

Definition 2.2 (Bag-Semantic Denotation). *For a query Q over schema $\mathcal{S}$ and a database instance D conforming to $\mathcal{S}$, we write $\{\!\{Q\}\!\}_D$ for the bag of result tuples produced by evaluating Q on D under SQL's bag semantics. Two queries Q_1 and Q_2 are* equivalent, *written $Q_1 \equiv Q_2$, if and only if $\{\!\{Q_1\}\!\}_D = \{\!\{Q_2\}\!\}_D$ for every instance D conforming to $\mathcal{S}$.*

Schema-level *integrity constraints* restrict the set of legal database instances and are essential for the soundness of many rewrite rules.

Definition 2.3 (Integrity Constraints). *We model four constraint types:* ***Primary Key*** *(pairwise distinct on key columns, no NULLs);* ***Foreign Key*** *(every non-NULL FK value appears in the referenced PK);* ***UNIQUE*** *(like PK but permits NULLs); and* ***NOT NULL*** *(column never takes* NULL*).*

2.2 Three-Valued Predicate Logic

SQL evaluates predicates under Kleene's strong three-valued logic (TRUE, FALSE, UNKNOWN), where UNKNOWN arises whenever an operand is NULL. Key dominance rules: $\mathsf{F} \wedge \mathsf{U} = \mathsf{F}$ and $\mathsf{T} \vee \mathsf{U} = \mathsf{T}$; negation preserves UNKNOWN. Filtering contexts (`WHERE`, `HAVING`, join `ON`) retain only rows evaluating to TRUE. Faithfully modeling three-valued semantics is essential, as many rewrite bugs stem from NULL corner cases [18]. Full truth tables appear in Appendix C.

2.3 Bounded Equivalence and Distinguishing Witnesses

Full semantic equivalence of SQL queries—over all possible database instances of unbounded size—is undecidable in general [8]. We therefore work with a *bounded* notion of equivalence that restricts attention to instances within a finite scope.

Definition 2.4 (Bounded Scope and Σ-Equivalence). *A* bounded scope $\Sigma = (K, \mathcal{D}, \nu)$ *fixes a maximum row count K per table, a finite value domain $\mathcal{D}$, and a NULL policy ν. Let* $\mathsf{Inst}(\Sigma, \mathcal{S})$ *be the set of all conforming instances with at most K rows per table. Queries Q_1, Q_2 are* Σ-equivalent *($Q_1 \equiv_\Sigma Q_2$) iff $\{\!\{Q_1\}\!\}_D = \{\!\{Q_2\}\!\}_D$ for every $D \in \mathsf{Inst}(\Sigma, \mathcal{S})$. A* distinguishing witness *is a D^* where the bags differ.*

The implementation achieves at-most-K coverage by encoding K symbolic rows per table with *present* flags (§5.1); a single Unsat at bound K implies Σ-equivalence. Bounded verification is *decidable*: the witness search reduces to a satisfiability query dispatched to Z3 [13].

2.4 CEGIS for Query Optimization

Counterexample-Guided Inductive Synthesis (CEGIS) [21, 36] is a paradigm from program synthesis in which a *synthesizer* proposes candidate programs and a *verifier* checks each candidate against a specification. When the verifier finds a counterexample—an input on which the candidate violates the specification—the counterexample is fed back to the synthesizer to prune the search space. The loop terminates when a candidate passes verification or the candidate space is exhausted.

We map this paradigm to query optimization as follows. The *specification* is the original query Q. The *candidate space* C is the set of rewrite candidates Q' produced by algebraic rules and LLM-assisted generation. The *verifier* is the bounded equivalence checker of Section 2.3, which either certifies $Q \equiv_\Sigma Q'$ or returns a distinguishing witness D^*. The optimization loop proceeds as:

1. **Generate** candidate rewrites C for Q.
2. **For each** $Q' \in C$: verify $Q \equiv_\Sigma Q'$.
   - UNSAT: certify Q', record cost.
   - SAT: reject Q'; use D^* to prune.
3. **Rank** certified rewrites by cost; **emit** cheapest.

Every accepted rewrite carries a replayable equivalence certificate; every rejected rewrite is accompanied by a concrete database instance demonstrating the semantic disagreement. We detail the full architecture in Section 3.

3 SYSTEM ARCHITECTURE

CertOpt is organized as a two-layer pipeline. *Layer A* generates rewrite candidates from the input SQL through a combination of

Algorithm 1: CertOpt CEGIS optimization loop.

Input: query Q, schema S, bound Σ
Output: (Q^*, π) or Q unchanged
$Q_{IR} \leftarrow \textsc{Parse}(Q)$ ▷ §3.2
$C \leftarrow \textsc{GenRewrites}(Q_{IR}, S) \cup \text{LLM}(Q, S)$ ▷ §4.1, §4.2
$F \leftarrow \textsc{ClassifyFamilies}(C)$ ▷ §4.3
$V \leftarrow \emptyset; R \leftarrow \emptyset$
foreach $c \in C$ **do**
  if $c.family \in$ *pruned* **then continue**
  if $\neg$ *StructuralVerify*(c, S) **then continue** ▷ §3.1
  $w \leftarrow \textsc{SynthesizeWitness}(Q_{IR}, c, S, \Sigma)$ ▷ §5–7.2
  if w=*UNSAT* **then** $V \leftarrow V \cup \{(c, \textsc{Cost}(c))\}$
  else if w=*SAT* **then** $R \leftarrow R \cup \{(c, w)\}$; $\textsc{Prune}(F, c)$
$Q^* \leftarrow \arg\min_{(c,cost) \in V}$ *cost*
return Q^* *with* Certificate *(§7.1) if* $\textsc{Cost}(Q^*) < \textsc{Cost}(Q_{IR})$, *else* Q_{IR}

rule-based algebraic transformations and LLM-assisted synthesis. *Layer B* subjects every candidate to a multi-stage safety filter that either certifies equivalence with a replayable proof artifact or rejects the candidate with a concrete distinguishing witness database. The two layers interact through a CEGIS feedback loop: witnesses produced by Layer B guide family-level pruning in Layer A, eliminating structurally related candidates without additional solver invocations.

3.1 Pipeline Overview

Given input query Q and schema S, the system parses Q into a typed `QueryIR` (§3.2). Layer A applies algebraic rewrite rules (§4.1) and, optionally, solicits LLM rewrites. Layer B processes each candidate through four stages: *(1) Structural verification*—schema-level checks reject candidates with unknown tables, type mismatches, or malformed grouping; *(2) Preprocessing*—predicate promotion and table elimination normalize the comparison surface; *(3) Witness synthesis*—the Z3-based checker (§5) searches for a distinguishing instance D^*; *(4) Witness validation*—every SAT witness is materialized in DuckDB (or SQLite) and both queries are executed to confirm disagreement (spurious witnesses are downgraded to UNKNOWN). Candidates passing all stages receive a cost estimate and a replayable equivalence certificate; the cheapest is selected as Q^*.

Algorithm 1 gives pseudocode for the full CEGIS optimization loop. In the pseudocode, SynthesizeWitness denotes the complete verifier pipeline including post-Sat witness validation and spurious-witness blocking (§7.2); a Sat return therefore indicates a *validated* distinguishing witness.

3.2 Typed Intermediate Representation

All rewrite rules and the verification engine operate on `QueryIR`, a Pydantic [12]-based typed abstract syntax tree that captures the relational structure of a SQL `SELECT` statement.

Expression nodes. The expression layer defines a closed hierarchy of node types: `ColumnRef`, `Literal`, `BinOp` and `UnaryOp` (arithmetic and logical operators), `AggCall` (with optional `DISTINCT`), `FuncCall` (e.g., `COALESCE`, `NULLIF`), `CaseExpr`, `InList` and `Between`, `WindowFunc` (with `PARTITION BY` and `ORDER BY`), `ExistsSubquery`, `InSubquery`, and `ScalarSubquery` (correlated and uncorrelated), and `Star` (`SELECT *`). Every node carries a *semantic type* (`INT`, `STRING`, `BOOL`, `DATE`, `FLOAT`) and a nullability flag inferred from schema `NOT NULL` constraints.

Query-level fields. A `QueryIR` instance comprises the following fields: `select` (list of named expressions), `from_table` (primary relation), `joins` (ordered list of join specifications, each with join type, right table, and `ON` predicate), `where` (filter predicate), `group_by` (grouping keys), `having` (post-aggregation filter), `order_by` (sort specifications with `ASC`/`DESC` and nulls placement), `limit` (row cap), `distinct` (boolean), `set_op` (`UNION`, `INTERSECT`, `EXCEPT` with optional `ALL`), and `set_right` (right operand for set operations). Subqueries are represented recursively as nested `QueryIR` nodes.

Normalization. Before verification, each `QueryIR` undergoes deterministic normalization: column references are fully qualified, aliases are lowercased, predicate conjuncts are sorted lexicographically, and commutative operators receive a canonical argument order. This normalization enables stable certificate caching—two syntactically different but logically identical `QueryIR` trees hash to the same canonical form—and simplifies structural comparison during family classification (§4.3).

Table 1: Design principles of CertOpt.

Principle	Rationale
Conservative over unsound	The system may reject truly equivalent rewrites (false SAT) but is designed never to accept a non-equivalent one.
Bounded, not complete	Equivalence proofs hold under bound Σ; the system provides a safety net, not a completeness guarantee.
Witness-grounded rejection	Every rejected rewrite is accompanied by a concrete database instance on which the original and rewritten queries disagree.
Certificate-carrying acceptance	Every accepted rewrite carries a replayable proof artifact that an independent checker can re-verify without re-running the solver.

3.3 Design Principles

Table 1 summarizes the four design principles that govern architectural decisions throughout CertOpt.

4 REWRITE GENERATION AND FAMILY PRUNING

Rewrite generation in CertOpt is intentionally *unsound by design*: the rules and the LLM are permitted to propose candidates that may change query semantics. Correctness is established exclusively by the verification layer (Layer B). This separation of concerns—generators propose, the verifier disposes—simplifies rule engineering and allows the system to absorb creative but unverified LLM suggestions safely.

Table 2: Rewrite rules in CertOpt. R1–R6 are algebraic transformations; M1–M3 are mutation rules for verifier stress testing.

ID	Rule	Preconditions
R1	Predicate pushdown	Inner joins only
R2	Predicate pullup	Inner joins only
R3	Join elimination	FK→PK, `NOT NULL` FK
R4	Redundant `DISTINCT` removal	Single table with PK
R5	Join reordering	≤6 permutations
R6	Projection minimization	Aggregation present
M1	`DISTINCT` toggle	—
M2	Aggregation swap	—
M3	Unwrap `ABS`	—

4.1 Algebraic Rewrite Rules

Table 2 lists the nine rewrite rules implemented in CertOpt. Rules R1–R6 are algebraic transformations drawn from standard query optimization theory; each is guarded by lightweight preconditions that restrict application to structurally appropriate queries. The three mutation rules (Distinct toggle, Aggregation swap, Unwrap ABS) are deliberately unsound perturbations designed to stress-test the verifier; they serve as negative controls during development and, in production, are disabled unless explicitly requested.

Each rule takes a `QueryIR` and the schema as input and returns zero or more candidate `QueryIR` instances. For example, predicate pushdown (R1) decomposes the `WHERE` clause into conjuncts, identifies those whose column references are fully covered by a single join's right table, and produces one candidate per eligible conjunct relocation. Join reordering (R5) enumerates permutations of the inner-join list up to a configurable cap (default six) to avoid combinatorial explosion on wide queries.

Crucially, the rules are *generators, not provers*: they emit syntactically plausible rewrites whose semantic equivalence is unknown until the verifier renders a verdict. This design permits aggressive rule application without risking silent correctness violations.

4.2 LLM-Assisted Generation

Beyond algebraic rules, CertOpt optionally solicits rewrite candidates from a large language model via a configurable LLM provider interface. The prompt supplies the original SQL text together with the full schema DDL (table definitions, column types, primary keys, foreign keys, and `NOT NULL` constraints), instructing the model to propose one or more semantically equivalent rewrites that may improve performance.

LLM outputs are parsed directly into a `QueryIR`; unparseable responses are discarded. An optional *schema-grounded repair* pass can canonicalize identifiers (resolving case mismatches and missing table qualifiers) and convert implicit comma-separated joins to explicit `JOIN . . . ON` syntax, though in practice the verifier catches all semantic errors regardless of surface-level canonicalization.

Every LLM-generated candidate enters the same Layer B safety filter as rule-generated candidates: the LLM contributes creativity while the verifier ensures that no semantically incorrect rewrite reaches the output.

4.3 Family Pruning

CertOpt amortizes verification cost through *family pruning*. Candidates are grouped by (*rule_id*, *structural_signature*), where the signature captures the syntactic shape of the transformation. When the verifier finds a witness D^* for one candidate, every candidate in the same family is pruned without an individual solver call (Algorithm 1, line 10), under the assumption that the witness targets the shared structural defect common to all family members.

Family pruning is a heuristic that only *skips* candidates that would be rejected; it cannot cause acceptance of a non-equivalent rewrite (candidates that survive are still individually verified at line 12). A falsely pruned candidate is a missed optimization, not a soundness violation.

On a BIRD benchmark [25] with 3,071 LLM-generated candidates, family pruning reduces solver calls by **33.1%** and solver time by 25.6%; see Appendix O for the full ablation.

5 BOUNDED EQUIVALENCE VERIFICATION

This section presents the core technical contribution: a faithful encoding of SQL bag semantics into quantifier-free SMT constraints. Given two queries Q_1, Q_2 over schema $\mathcal{S}$ with bounded scope $\Sigma = (k, \mathcal{D}, v)$, CertOpt constructs a formula Φ such that Φ is satisfiable if and only if there exists a database instance $D \in \Sigma$ on which $[\![Q_1]\!]_D \neq [\![Q_2]\!]_D$ under bag semantics.

5.1 Symbolic Database Construction

The verification engine uses an *at-most-k* encoding: for a configured maximum bound K, it allocates K symbolic rows per table, each carrying a Boolean *present* flag. Rows with *present* $= \bot$ represent absent rows and are excluded from query evaluation, so a single solver call at bound K covers *all* instances with at most K rows per table, including 0-row tables and mixed-cardinality instances. Semantically, the k=K call alone suffices for full at-most-K coverage; the ascending schedule $k = 1, 2, \ldots, K$ is purely an operational optimization that enables early Sat short-circuiting and partial results when a higher-k call times out.

For each table T, the engine allocates K symbolic rows. Each cell is a pair of Z3 variables:

Definition 5.1 (Nullable Cell). *A* nullable cell *is a pair* $\langle$*is_null* : Bool, *val* : Sort$\rangle$ *where* Sort *is determined by the column's semantic type. When is_null is true, the cell represents SQL* `NULL`*; otherwise, val carries the concrete value.*

Type encoding. All types are mapped to finite Z3 domains. Integers and decimals use RealSort with an integrality constraint (IsInt) for integer columns, bounded to an interval $[\ell, h]$ widened to include all numeric literals appearing in Q_1 and Q_2. Strings are encoded as integer *ranks* drawn from a sorted symbol table of n_s canonical symbols: each string variable is constrained to $[0, n_s)$, and the integer ordering preserves lexicographic order among the symbols. Dates and timestamps use the same rank-based encoding. Booleans are constrained to $\{0, 1\}$.

Integrity constraints. The symbolic database must respect the schema's declared constraints.

- *Primary key and unique.* For each PK or UNIQUE column group $(c_{i_1}, \ldots, c_{i_p})$, we assert pairwise tuple inequality across

all $\binom{k}{2}$ row pairs: if both rows have all group columns non-null, then at least one column value must differ. This requires $O(k^2)$ constraints per group.
- *Foreign key (child → parent).* For each FK constraint $T_s.c_s \to T_d.c_d$, and for every symbolic row r of T_s, we assert

$$r.c_s.null \ \vee \ \bigvee_{r' \in T_d} \left(\neg r'.c_d.null \wedge r.c_s.val = r'.c_d.val\right)$$

encoding existential referencing.
- *Reverse FK guard.* When the source column of a FK is itself a primary key (a parent-side back-reference), the constraint is *suppressed*. In a bounded-k encoding, forcing every parent row to match some child row would render nullable FK columns effectively non-null, producing false equivalence verdicts. Only the child→parent direction is enforced.
- *Not-null.* For every column declared `NOT NULL`, we assert $\neg is\_null$.

5.2 Three-Valued Predicate Evaluation

SQL predicates evaluate under Kleene's strong three-valued logic (3VL): any predicate may yield True, False, or Unknown. We encode this as follows.

Definition 5.2 (TriBool). *A* TriBool *is a pair* $\langle is\_unknown : \text{Bool},\ val : \text{Bool}\rangle$. *When is_unknown is true, the predicate result is* Unknown*; otherwise, val carries the definite* True *or* False *value.*

The SQL connectives are encoded with the standard 3VL dominance rules:

- **AND.** False dominates Unknown. Let $af = \neg a.is\_unk \wedge \neg a.val$ and analogously for b. Then: $is\_unk = \neg(af \vee bf) \wedge (a.is\_unk \vee b.is\_unk)$; $val = a.val \wedge b.val$.
- **OR.** True dominates Unknown. Let $at = \neg a.is\_unk \wedge a.val$ and analogously for b. Then: $is\_unk = \neg(at \vee bt) \wedge (a.is\_unk \vee b.is\_unk)$; $val = a.val \vee b.val$.
- **NOT.** is_unk is unchanged; val is negated.
- **SQL equality with NULLs.** `NULL = NULL` → Unknown; `NULL` $= x \to$ Unknown; $x = y \to$ True/False.

Filtering contexts. In SQL, `WHERE`, `HAVING`, and `ON` clauses retain a row only when the predicate is *definitely* True. We define:

$$tb\_true(tb) \ = \ \neg tb.is\_unknown \wedge tb.val$$

This single predicate is used as the survival condition in all filtering contexts, ensuring that rows with Unknown predicates are correctly discarded.

5.3 Expression and Predicate Encoding

Scalar expressions are evaluated recursively to produce *NullableVal* results (Definition 5.1). The encoding handles the following constructs:

Arithmetic. Binary operations ($+, -, \times, /$) propagate NULLs: if either operand is null, the result is null. Otherwise, the result value is computed over Z3 arithmetic with exact Real division.

Comparison. Each comparison ($=, \neq, <, \leq, >, \geq$) yields a TriBool. If either operand is null, the result is Unknown; otherwise the result is definite.

LIKE. Literal patterns are matched against the finite symbol table, producing an exact disjunction of equalities. Non-literal patterns fall back to a conservative approximation.

Function calls. `COALESCE`, `NULLIF`, `ABS`, and `CAST` are given exact encodings. `COALESCE`$(e_1, \ldots, e_n)$ is encoded as a right fold: the result takes the value of the first non-null argument. `CAST` to `BOOLEAN` maps nonzero to 1 and zero to 0. All other function calls are encoded as fresh unconstrained Z3 variables (§5.8).

CASE expressions. Each `WHEN` branch is evaluated sequentially under 3VL. The first branch whose condition is definitely True determines the result. If no branch fires, the `ELSE` value (or `NULL` if absent) is returned.

IN-list and BETWEEN. e `IN` $(v_1, \ldots, v_n)$ is expanded to an OR chain with 3VL semantics: $e = v_1 \vee \cdots \vee e = v_n$. e `BETWEEN` a `AND` b is expanded to $(e \geq a) \wedge (e \leq b)$.

Subqueries. `EXISTS` subqueries are encoded as the disjunction of the inner query's surviving rows: $\bigvee_j inner\_survives_j$. `IN` subqueries produce a 3VL membership test analogous to IN-list expansion over the inner result rows. Scalar subqueries return the value of the first surviving inner row via a nested If chain; if no row survives, the result is `NULL`.

5.4 Join Evaluation

CertOpt evaluates joins by materializing the Cartesian product of row indices $[0, k)$ across all participating tables and then filtering according to join type.

Definition 5.3 (Combo). *A* combo *is a mapping* $c : \mathcal{A} \to [0, k)$ *from table aliases* $\mathcal{A}$ *to row indices. The full Cartesian product yields* $k^{|\mathcal{A}|}$ *combos.*

Joins are processed left-to-right, accumulating surviving combos at each step:

- **INNER JOIN.** A combo survives if and only if the `ON` predicate is definitely True (via *tb_true*).
- **LEFT OUTER JOIN.** For each left-side row i, we evaluate the `ON` predicate against every right-side row j. Matched pairs (i, j) where `ON`(i, j) is True produce output rows. If $\neg\exists j$ such that `ON`(i, j) is True, a single *NULL-padded* row is emitted: the right-side columns are all set to $\langle is\_null = \top, val = 0\rangle$.
- **RIGHT OUTER JOIN.** Symmetric to LEFT: unmatched right-side rows produce NULL-padded left columns.
- **FULL OUTER JOIN.** Combines LEFT and RIGHT: both unmatched-left and unmatched-right NULL-padded rows are emitted.
- **CROSS JOIN.** All combos survive unconditionally (no `ON` predicate).

Multi-join queries are handled by iterative composition: the result of joining tables $T_1, \ldots, T_i$ becomes the "left side" for the $(i{+}1)$-th join. This left-to-right recursive evaluation correctly handles chains of outer joins (e.g., A `LEFT JOIN` B `LEFT JOIN` C) without duplicating matched combos.

5.5 Aggregation and GROUP BY

Aggregation is encoded over the set of surviving combos from the join phase. Let $n = k^{|\mathcal{A}|}$ be the total number of combos.

Definition 5.4 (Same-Group Matrix). *The* same-group matrix $sg \in \text{Bool}^{n \times n}$ *is defined by* $sg[i][j] = \top$ *iff combos i and j agree on all GROUP BY keys under SQL equality (both-null-or-both-equal per*

key column). When no GROUP BY is present but aggregation is used (scalar aggregate), $sg[i][j] = \top$ for all i, j.

Definition 5.5 (Group Representative). *Combo i is a* group representative *iff it survives filtering and no earlier-indexed surviving combo belongs to the same group:*

$$is\_rep[i] = survives[i] \ \wedge \ \neg \bigvee_{j<i} \big(survives[j] \wedge sg[i][j]\big).$$

Each output row of the aggregated query corresponds to exactly one group representative.

Aggregate functions are computed over the group of each representative. Let $G_i = \{j : survives[j] \wedge sg[i][j]\}$. Then: Count(∗)=$|G_i|$; Count($e$)=$|\{j \in G_i : \neg e_j.null\}|$; Sum($e$)=$\sum_{j \in G_i, \neg e_j.null} e_j.val$ (NULL if all null); Avg=Sum/Count with exact Real division; Min/Max(e): non-null value in G_i with no strictly smaller (resp. larger) peer; Count(Distinct e): count j only if it is the first non-null occurrence of $e_j.val$ in G_i. Integer truncation differences are caught by witness validation (§7).

Empty-group behavior. Count on an empty group returns 0; all other aggregates return NULL, matching the SQL standard.

HAVING. The HAVING predicate is evaluated as a TriBool over the aggregate values of each group representative. Only groups whose HAVING result is definitely True survive.

Scalar aggregates. When no GROUP BY is present, all surviving combos are treated as a single group. The query produces exactly one output row (the scalar aggregate result), even if the input is empty.

5.6 ORDER BY and LIMIT

ORDER BY alone is cosmetic for bag equality and is not modeled. When combined with LIMIT ℓ, CertOpt assigns each surviving row a rank $rank(r) = 1 + |\{s : s \prec_{ord} r\}|$ and retains rows with $rank \le \ell$. The comparison $\prec_{ord}$ is NULL-aware (NULLS LAST for ASC, NULLS FIRST for DESC). Ties are broken by unconstrained integer variables, letting Z3 existentially quantify over all tie resolutions. See Appendix D for the full encoding.

5.7 Difference Predicate

The final constraint asserts that the two query results disagree under bag semantics.

Definition 5.6 (Bag Disagreement). *Let R_1, R_2 be the result-row lists of Q_1, Q_2. The* difference predicate *asserts the existence of a tuple whose* multiplicity *differs:*

$$diff \ = \bigvee_{r \in R_1 \cup R_2} \Big(survives(r) \ \wedge \ count(r, R_1) \neq count(r, R_2)\Big)$$

where $count(r, R) = \sum_{r' \in R} \mathrm{If}(survives(r') \wedge match(r, r'),\ 1,\ 0)$.

Tuple matching. Two result rows *match* iff every projected column satisfies *both-null-or-both-equal*: $match(r, r') = \bigwedge_\ell \big((v_\ell.is\_null \wedge v'_\ell.is\_null) \vee (\neg v_\ell.is\_null \wedge \neg v'_\ell.is\_null \wedge v_\ell.val = v'_\ell.val)\big)$. If R_1 and R_2 have different arity (different numbers of projected columns), matching always fails and *diff* is trivially satisfiable.

DISTINCT queries. For queries with SELECT DISTINCT, the multiplicity is capped at 1: $count(r, R) \leftarrow \mathrm{If}(count(r, R) \ge 1,\ 1,\ 0)$, converting bag semantics to set semantics for that branch. The two branches may have different DISTINCT flags.

5.8 Conservative Under-Approximation

Not every SQL construct admits an exact SMT encoding within the bounded framework. CertOpt handles unmodeled constructs via a principled *conservative under-approximation* that preserves a key safety invariant.

Definition 5.7 (Fresh-Variable Encoding). *For any subexpression e that CertOpt cannot model exactly (e.g., an unsupported function call or a window function beyond basic Row_Number), the encoder replaces e with a fresh, unconstrained Z3 variable $\hat{v}_e$. Crucially, each occurrence is keyed by the Python object identity of the AST node: distinct occurrences of syntactically identical subexpressions in Q_1 and Q_2 receive* different *fresh variables.*

Proposition 5.1 (Safety of Fresh-Variable Encoding). *Let Φ be the SMT formula produced by CertOpt and Φ^* be the (hypothetical) exact encoding. Then:*

1. *If Φ is Unsat, then Φ^* is Unsat (no false equivalence claims).*
2. *If Φ is Sat, Φ^* may or may not be Sat (false witnesses are possible).*

The proof appears in Appendix A.3.

LIKE patterns. As noted in §5.3, LIKE with a literal pattern is encoded *exactly* via symbol-table matching. Only non-literal LIKE patterns use the equality fallback, which is a predicate *strengthening* (not a relaxation) and therefore does *not* satisfy the fresh-variable relaxation argument. To maintain soundness, CertOpt treats any Unsat verdict that depends on a non-literal LIKE approximation as Unknown.

Validation catch. Every Sat witness is materialized into an in-memory DuckDB database (falling back to SQLite when DuckDB cannot execute the dialect), both queries are executed, and their results are compared as multisets. If the results agree, the witness is *spurious* and the verdict is downgraded from Neq to Unknown; if neither engine can execute the queries, the verdict is conservatively downgraded to Unknown. This execution-based validation ensures that approximations introduced by fresh-variable encoding do not produce false rejection of a correct rewrite.

6 COMPOSITIONAL VERIFICATION

Monolithic bounded verification does not scale to wide queries. The JOB-Complex benchmark contains queries with 6–16 tables; at bound k=2, a 10-table query generates $2^{10} = 1{,}024$ row combinations in a single Cartesian product. Even aggressive preprocessing—table elimination, predicate promotion—cannot reduce the combinatorial explosion for queries in which every table participates in at least one semantically relevant join or predicate. We therefore propose *compositional per-join decomposition*: instead of encoding the entire query pair monolithically, CertOpt isolates each structurally changed join into a small *local region*, verifies each region independently, and composes the local verdicts into a whole-query decision.

6.1 Per-Join Decomposition

For predicate pushdown and pullup rewrites (rules R1 and R2, Table 2) on INNER joins, CertOpt applies the following seven-step decomposition algorithm:

Step 1: Detect moved predicates per join. Canonically compare the WHERE conjuncts and JOIN ON conjuncts between the original query Q and rewrite Q'. Each predicate that appears in WHERE of one query and in the ON clause of a specific join in the other (or vice versa) is classified as a *moved predicate* and associated with the join to or from which it migrated.

Steps 2–4: Local region construction. For each changed join J with moved predicate φ, the *local alias set* contains T plus all aliases referenced by structural ON predicates of J, capped at $|local| \leq 6$. *Boundary aliases*—those referenced outside the local region (in SELECT, GROUP BY, other ON clauses)—define the interface schema I, which exports *all* columns of each boundary alias to preserve bag multiplicities.

Steps 5–7: Verify and compose. Build a standalone query pair per local region (φ in WHERE vs. ON), both selecting I. Verify each pair via SynthesizeWitness() with an adaptive bound (k=1 for regions with ≥5 tables, global k otherwise). If every region returns Unsat, the whole-query rewrite is accepted; any Sat or Unknown yields an *inconclusive* whole-query verdict (not rejection).

Validity conditions. Decomposition requires: (i) rule R1 or R2; (ii) INNER joins only; (iii) top-level conjunct predicates; (iv) ≤2 aliases per predicate; (v) $|local| \leq 6$; (vi) pairwise disjoint local alias sets across regions; and (vii) no subqueries, window functions, or outer joins.

6.2 Composition Theorem

Theorem 6.1 (INNER JOIN Predicate Relocation). *Let Q contain a sub-expression $L \bowtie_\theta R$ under an* INNER JOIN*, and let φ be a predicate referencing only attributes of R (or L, or both). Let I denote the schema consisting of all columns of the boundary aliases. If, for every database instance D within bound Σ, the bag of I-projected tuples produced by $(L \bowtie_\theta R)$* WHERE *φ equals that produced by $L \bowtie_{(\theta \wedge \varphi)} R$, then replacing one sub-expression by the other in Q preserves the bag-semantic result of Q in any outer context that consumes only I.*

The phrase "consumes only I" is formalized by two *admissibility conditions* on the outer context: (i) every reference to an alias inside the local block appears only via columns in I, and (ii) predicates in the outer context may not reference internal aliases not exposed through I. The full proof treats the local block as a derived relation with schema I and shows bag-contextual substitutivity under these conditions; see Appendix A.6.

Corollary 6.1 (Compositional Soundness). *If all n local regions of a decomposed rewrite return* Unsat *and the decomposition satisfies the validity conditions of §6.1, then the full query rewrite is equivalent under bound $\Sigma' = (k', \mathcal{D}, \nu)$ where $k' = \min_i k_i$ and $\Sigma_i = (k_i, \mathcal{D}, \nu)$ is the bound used for region i. (When all regions use the same global bound Σ, this reduces to full Σ-equivalence.) If any local region returns* Sat*, the result is* inconclusive*—the local witness may not survive the outer context, so it* ***cannot*** *be used for rejection.*

This asymmetry is the central safety property: compositional verification is used only for *acceptance* (all-local-UNSAT ⇒ accept), never for rejection.

Multiple regions. When n predicates are moved across n distinct joins, CertOpt proves each local region independently and composes the results by repeated substitution: $Q_0 \to Q_1 \to \cdots \to Q_n$. Each step replaces exactly one local block with its verified-equivalent variant, preserving the semantics established in all previous steps.

Table 3: Composition decision rules for per-join decomposition.

Local Result	Decision	Rationale
Unsat	Accept	Local equiv. under $\Sigma' \Rightarrow$ global equiv. under Σ'
Sat	Inconclusive	Local witness may not survive outer context
Unknown	Inconclusive	Fall back to monolithic

Soundness rules. Table 3 summarizes the composition logic.

6.3 Complexity Analysis

We now formally characterize the combinatorial reduction achieved by per-join decomposition.

Proposition 6.1 (Combinatorial Reduction). *Let Q be a query over n tables verified at bound k. Monolithic verification enumerates k^n row combinations. If the rewrite moves m predicates across m distinct joins, each producing a local region of n_i tables ($i = 1, \ldots, m$), then compositional verification enumerates $\sum_{i=1}^{m} k^{n_i}$ combinations in total.*

The reduction factor in *formula size* is at least $k^{n-n_{\max}}/m$, which grows exponentially in the gap between total table count and largest local region size. (Actual wall-clock speedup may differ due to solver heuristics.) See Appendix A.7 for the full proof.

Example. For JOB-C01 (n=9, m=5, $n_i \in \{1, 2, 2, 2, 2\}$, k=2): monolithic enumerates $2^9 = 512$ combos; compositional enumerates only 2+4+4+4+4 = 18, a 28× formula-size reduction. At k=3 the gap widens to 19,683/39 ≈ 505×. Across the 27 verified JOB-Complex queries, the median reduction is 85×.

6.4 Scaling Results

Without compositional decomposition, monolithic verification collapses entirely on JOB-Complex at k≥4 (0/30 verified; see ablation A5 in §8.5). With per-join decomposition, CertOpt verifies **27 of 30** queries at k=3 in 492 s. The 3 unverified queries (C05, C09, C20) have local regions exceeding the $|local| \leq 6$ threshold due to transitive ON dependencies.

7 CERTIFICATES AND SOUNDNESS

A proof-carrying optimizer must not only decide equivalence but also *explain* its decisions in a form that can be independently audited. CertOpt accompanies every accepted rewrite with a replayable *equivalence certificate* and every rejected rewrite with a concrete *distinguishing witness*. This section defines the certificate format, describes the witness validation pipeline that eliminates spurious counterexamples, and states the three soundness theorems that underpin the system's guarantees.

7.1 Certificate Structure

An equivalence certificate π accompanies every accepted rewrite and contains sufficient information for an independent checker to reproduce the verification verdict:

$$\pi \;=\; \big(\Sigma,\; I_{norm},\; K,\; status,\; Q_{orig},\; Q_{rewrite},\; h_{IR},\; h_Q,\; h_S\big)$$

Here Σ is the bounded scope (k, domain, NULL policy); I_{norm} is the normalized IR after canonical ordering; K is the proven cardinality bound (highest k yielding Unsat); *status* records monolithic vs. compositional verification; Q_{orig} and $Q_{rewrite}$ are the rendered SQL texts; and h_{IR}, h_Q, h_S are cryptographic hashes of the IR, SQL, and schema catalog for tamper detection.

Replayability. The certificate is *replayable*: an independent checker can reconstruct the bounded scope Σ from the certificate, re-parse both queries, re-invoke SynthesizeWitness(), and confirm that the solver returns Unsat. The schema hash h_S detects schema drift—if the catalog has changed since certification, the certificate is invalidated and re-verification is required. We validated this claim by generating and replaying 500 equivalence certificates from the VeriEQL LeetCode benchmark with a 100% pass rate (Appendix N).

7.2 Witness Validation

When witness synthesis returns Sat, CertOpt does not immediately reject the rewrite. Conservative approximations (§5.8) may produce *spurious witnesses* on which the queries actually agree. A three-stage pipeline eliminates false positives: *(1)* materialize the Z3 model into a DuckDB (or SQLite) database, *(2)* execute both queries and compare result multisets, and *(3)* if results agree, downgrade the verdict to Unknown. If a spurious witness is detected, a CEGAR-style blocking constraint excludes it and the solver is re-invoked (up to 3 iterations). When every bound in the at-most-k schedule produces only spurious witnesses (no genuine Unsat at any k), the verdict remains Unknown.

7.3 Soundness Theorems

We state three theorems that establish the formal guarantees of CertOpt's verification engine. Two mechanisms ensure that theorem preconditions are satisfied whenever Unsat is reported: (i) *exact-fragment classification*—the encoder tracks whether each subexpression is modeled exactly or via fresh-variable relaxation (Definition 5.7), routing the verdict through Theorem 7.1 or Theorem 7.2 accordingly; and (ii) *bounded-k completeness guards* (Appendix M)—when a guard detects that the encoding may be incomplete at the current bound (e.g., `HAVING COUNT(*) >= N` with $N > k$), the engine returns Unknown rather than Unsat.

Theorem 7.1 (Rewrite Soundness). *Let Q, Q' be queries whose encoding lies within the exact fragment $\mathcal{F}_{\text{exact}}$ (Table 11). If SynthesizeWitness returns* Unsat *for (Q, Q', S, Σ), then $Q \equiv_\Sigma Q'$: for all instances D conforming to S with $\le k$ rows per table and values in $\mathcal{D}$, $[\![Q]\!]_D = [\![Q']\!]_D$.*

Theorem 7.2 (Conservative Approximation Safety). *For any construct encoded via a* value relaxation*—i.e., replacing a value expression with a fresh unconstrained variable of matching sort—if SynthesizeWitness returns* Unsat*, then $Q \equiv_\Sigma Q'$. This does not apply to predicate-strengthening approximations (e.g., non-literal* `LIKE` *falling back to equality), which may shrink the solution space; such cases yield* Unknown.

Theorem 7.3 (Bounded Optimality). *The output Q^* of Algorithm 1 satisfies: (1) $Q^* \equiv_\Sigma Q$; and (2) $\text{cost}(Q^*) \le \text{cost}(Q')$ for all $Q' \in C_{\text{eq}}(Q)$, the set of explored candidates verified equivalent under Σ.*

Proofs of all three theorems appear in Appendix A.2–A.5.

8 EXPERIMENTAL EVALUATION

8.1 Setup

Implementation. CertOpt is implemented in Python (~23 K lines of code across 46 source files) using Z3 [13] as the SMT back-end, `sqlglot` [27] for SQL parsing, and a Pydantic [12]-based typed intermediate representation (§3.2). The implementation is covered by 508 unit tests exercising every encoding path, rewrite rule, and witness-validation pipeline stage.

Hardware. All experiments run on an Apple M3 Max (14 cores, 36 GB RAM) under macOS, single-threaded, with Z3 4.13.

Benchmarks. We evaluate along three axes:

(1) **Axis 1 (Verifier accuracy):** Three suites from the VeriEQL benchmark [19]: Calcite (397 pairs from the Apache Calcite optimizer test suite), Literature (64 pairs from SQL textbooks and research papers), and LeetCode (23,994 pairs from competitive-programming SQL problems).
(2) **Axis 2 (Rewrite validation):** 30 JOB-Complex queries over IMDB (5–16 tables per query) [24, 40].
(3) **Axis 3 (LLM guardrail):** 24,495 SQLStorm query pairs [35] over JOB, TPC-H, TPC-DS, and StackOverflow, in two modes: verifying rule-based rewrites and vetting LLM-generated rewrites.

Baseline. Our primary accuracy baseline is VeriEQL [19] (OOPSLA '24), the current state-of-the-art bounded SQL equivalence verifier. VeriEQL uses a constraint-based encoding over uninterpreted sorts with integrity constraints and runs at bound k=32 by default.

Configuration. Unless stated otherwise, CertOpt uses bound k=3 with an at-most-k dense schedule (rows in $[1..k]$), a per-pair solver timeout of 10 s, and mandatory post-SAT witness validation (§3.1). Column-order normalization is disabled on the VeriEQL benchmarks to match VeriEQL's strict positional semantics.

8.2 Axis 1: Verifier Accuracy (VeriEQL Benchmarks)

Table 4 summarizes the main results across all three VeriEQL suites. CertOpt achieves a **zero false equivalence claim** rate: every pair classified as EQU is confirmed correct by cross-comparison with VeriEQL and by independent execution on DuckDB and SQLite.

Cross-tabulation. Table 5 gives the full verdict cross-tabulation against VeriEQL on all three suites combined (24,455 pairs).
Four results stand out:
(1) Zero false equivalence claims. The *False EQU* column is 0 across all three suites: every pair classified as EQU is confirmed

Table 4: Axis 1 main results. *False EQU* counts pairs where CertOpt claims equivalence but VeriEQL refutes it. *Speedup* is VeriEQL wall time divided by CertOpt wall time. PF = parse failure.

Suite	Pairs	EQU	NEQ	UNK	PF	False EQU	Speedup
Calcite	397	346	2	49	0	0	582×
Literature	64	30	18	15	1	0	286×
LeetCode	23,994	15,581	3,581	4,635	197	0	140×
Total	**24,455**	**15,957**	**3,601**	**4,699**	**198**	**0**	—

Table 5: Cross-tabulation of CertOpt verdicts vs. VeriEQL verdicts (Calcite + Literature + LeetCode, 24,455 pairs). NSE = Not Supported by Encoding; ERR = internal error or unsupported syntax.

	VeriEQL verdict				
CertOpt	EQU	NEQ	NSE	ERR	**Total**
EQU	12,318	719	1,333	1,587	15,957
NEQ	241	2,090	242	1,028	3,601
UNK	2,634	806	388	871	4,699
PARSE_FAIL	7	4	5	182	198
Total	15,200	3,619	1,968	3,668	**24,455**

correct by cross-comparison with VeriEQL and by independent execution on DuckDB and SQLite. (This guarantee is relative to the bounded scope Σ, the exact-fragment / conservative-safety classification of §5.8, and the bounded-k completeness guards of §7.3.)

(2) Validated non-equivalence. On LeetCode, 241 pairs receive NEQ from CertOpt where VeriEQL reports EQU. Every one of these 241 witnesses is *confirmed* by executing both queries on the witness database in DuckDB and SQLite—the results genuinely differ. These are VeriEQL false equivalence proofs, not our false rejections (§8.2).

(3) Broader coverage. VeriEQL returns NSE or ERR on 5,636 of 24,455 pairs. CertOpt renders a definitive verdict (EQU or NEQ) on 4,190 of these—a 74.3% coverage gain on the "undecidable" pairs. The advantage comes from broader SQL dialect support: `FILTER(WHERE)`, `VALUES`, `FETCH FIRST`, `IS [NOT] DISTINCT FROM`, and derived-table column aliases (§3.2).

(4) Significant speedup. CertOpt completes the Calcite suite in 157.1 s (mean 395.4 ms, median 135.6 ms, p_{95} 1,191.2 ms)—a **582×** speedup over VeriEQL's 91,501 s. On LeetCode (23,994 pairs), CertOpt finishes in 8.7 h (mean 1,301 ms, median 184 ms) vs. VeriEQL's 1,219 h—a **140×** speedup. Figure 1(b) shows the per-pair solve-time CDF: over 50% of pairs complete in under 200 ms. The speedup derives from four sources: (i) CertOpt runs at k=3 rather than k=32, yielding a smaller SMT formula; (ii) the at-most-k dense encoding checks all cardinalities $1 \le k' \le k$ in a single solver call, avoiding the combinatorial blow-up of VeriEQL's fixed-k instances; (iii) an adaptive schedule that starts at k=1 and escalates only when needed—most pairs are decided at $k \le 2$; and (iv) preprocessing

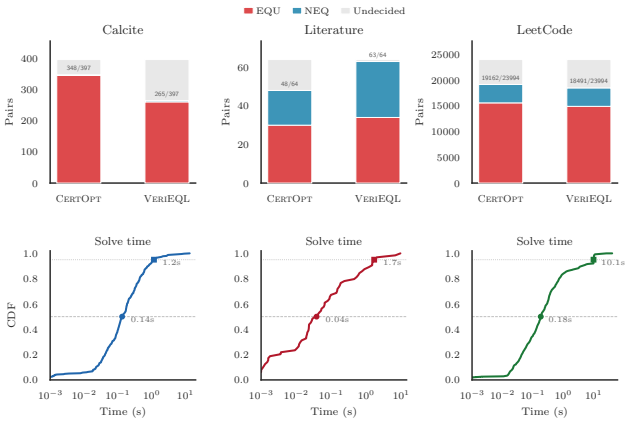

Figure 1: Top row: decided pairs (EQU + NEQ) per suite, with own y-scale. Numbers above bars show decided / total. Grey segments are undecided (UNK, parse failures, or VeriEQL NSE/ERR). Bottom: per-pair solve-time CDF (log scale).

passes (§F) that reduce the number of tables before the solver sees the formula.

Agreement analysis. On the 15,200 pairs where VeriEQL reports EQU, CertOpt agrees on 12,318 (81.0%) and returns UNKNOWN on 2,634 (17.3%). These UNKNOWN verdicts arise because the bounded SMT encoding over-approximates three aspects of SQL semantics: (i) exact rational arithmetic where SQL uses floating-point (36%), (ii) nondeterministic row selection in scalar subqueries and `LIMIT` (29%), and (iii) fresh unconstrained variables for functions outside the modeled fragment such as `DATE()` and window functions (28%). In each case the solver finds a satisfying assignment, but executing both queries on the witness database produces identical results, so the witness is spurious and CertOpt conservatively withholds judgement (Appendix P). The 241 NEQ-vs-EQU pairs are confirmed VeriEQL false equivalence proofs (§8.2).

VeriEQL false equivalence proofs found. Our cross-comparison uncovered **241 confirmed VeriEQL false equivalence proofs** on LeetCode: every witness was validated in DuckDB and SQLite. Additionally, 3 false *refutations* on Calcite/Literature—cases where VeriEQL claims NEQ but the queries are in fact equivalent—were identified and confirmed (Appendix G.7). Five root causes account for all 241 false equivalence proofs: *(i)* integer division encoded as real division (153 pairs); *(ii)* conflicting PK constraints collapsing semantic distinctions (30 pairs); *(iii)* column-pivot NULL/duplicate semantics in `CASE`/`LEFT JOIN` rewrites (34 pairs); *(iv)* `NOT IN` with NULL propagation (15 pairs); *(v)* `IF()` and nested `CASE` encoding gaps (9 pairs).

8.3 Axis 2: Rewrite Validation (JOB-Complex)

Table 6 summarizes the JOB-Complex results. CertOpt generates 2 rule-based rewrite candidates per query (predicate pushdown R1 and pullup R2) and verifies each via compositional per-join decomposition (§6).

Table 6: JOB-Complex results (30 queries, 6–16 tables). *Syntactic cost* uses the heuristic from Appendix I. DuckDB execution is median wall-clock over 3 runs on the full IMDB database.

	Verified	Unverified	Total
Queries	27	3	30
Avg tables	10.2	15.0	10.7
Syntactic cost red.	7.8%	—	—
DuckDB exec. speedup	1.00×	—	—
Median verify time	4.4 s	0.1 s	—

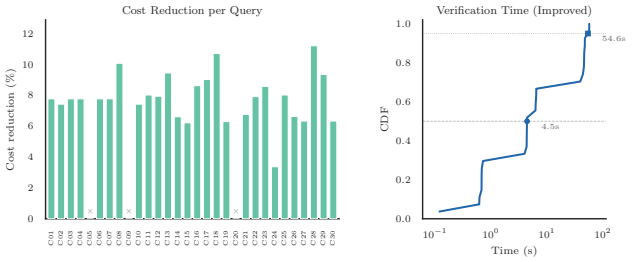

Figure 2: Left: per-query cost reduction (%). Grey bars are the 3 unverified queries (C05, C09, C20). Right: verification-time ECDF for the 27 improved queries (log scale).

Coverage. Of the 30 JOB-Complex queries, **27** (90%) are successfully verified and improved via compositional decomposition. Without decomposition, monolithic verification cannot decide *any* of the 30 queries—every query exceeds the combinatorial safety limit (§6.4).

Cost and execution. Verified rewrites achieve a mean syntactic cost reduction of 7.8%. DuckDB execution on the full IMDB database shows 1.00× median speedup, confirming that modern optimizers already apply equivalent transformations internally. The primary value is *proving rewrite correctness*: each certified rewrite carries a replayable equivalence certificate.

Unverified queries and timing. The 3 unverified queries (C05, C09, C20, 14–16 tables) exceed the $|local| \leq 6$ region threshold. The full benchmark completes in 492 s (median 4.4 s, p_{95} 57.4 s per query). Figure 2 shows cost reductions and timing.

8.4 Axis 3: LLM Guardrail (SQLStorm)

We evaluate CertOpt on SQLStorm [35] workloads across four schemas (TPC-H, TPC-DS, StackOverflow, JOB) in two modes: (a) verifying SQLStorm's own *cross-database compatibility* rewrites, and (b) vetting *LLM-generated* semantically-equivalent rewrites as a guardrail against silent semantic regressions. We evaluate 12,705 rule-based and 11,790 LLM-generated rewrite pairs (24,495 total) at bound k=2 with a 10 s per-pair timeout and mandatory witness validation.

Option 1: Verifying rule-based rewrites. Table 7 (top) shows the compatibility-rewrite results: **7,238** of **12,705** pairs are verified equivalent (EQU) with only **8** NEQ. The low rejection rate confirms

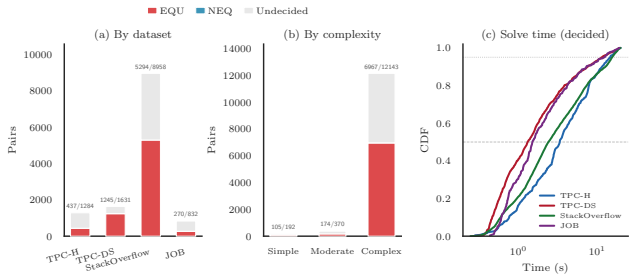

Figure 3: SQLStorm compatibility-rewrite evaluation (Option 1). (a) Decision counts by dataset. (b) By query complexity. (c) Solve-time CDF for decided pairs (log scale). Numbers above bars show decided/total.

Table 7: Axis 3: SQLStorm evaluation (24,495 pairs total). Option 1 verifies SQLStorm's cross-database compatibility rewrites; Option 2 vets LLM-generated rewrites. *Rej %* = NEQ/(EQU+NEQ). All NEQ witnesses are validated.

Option	Dataset	Total	EQU	NEQ	UNK	TMO	PF	Rej %
Rule-based	TPC-H	1,284	437	0	249	498	100	0%
	TPC-DS	1,631	1,237	8	129	165	92	0.6%
	StackOverflow	8,958	5,294	0	1,128	2,127	409	0%
	JOB	832	270	0	105	114	343	0%
	Subtotal	**12,705**	**7,238**	**8**	**1,611**	**2,904**	**944**	**0.1%**
LLM	TPC-H	1,266	499	6	367	364	30	1.2%
	TPC-DS	1,622	1,103	68	267	149	35	5.8%
	StackOverflow	8,079	2,179	36	3,868	1,672	324	1.6%
	JOB	823	295	21	262	208	37	6.6%
	Subtotal	**11,790**	**4,076**	**131**	**4,764**	**2,393**	**426**	**3.1%**

that SQLStorm's compatibility pass is highly reliable. Parser coverage is the main limitation: 944 pairs (7.4%) fail on Postgres-specific syntax not yet supported. Figure 3 shows per-dataset decision rates.

Option 2: LLM rewrite guardrail. We prompted an LLM to produce structurally different but semantically-equivalent rewrites of SQLStorm source queries (e.g., subquery↔JOIN, CTE inlining, predicate pushdown). Table 7 (bottom half) shows the results across 11,790 rewrite pairs.

Three findings stand out:

(1) LLM rewrites are frequently wrong. CertOpt rejects **131 of 4,207** decided LLM rewrites (3.1% overall rejection rate) with validated witnesses. TPC-DS exhibits the highest rate (5.8%: 68 of 1,171 decided pairs), where the dominant failure mode is the LLM restructuring explicit `JOIN. . . WHERE` into `WITH. . . EXISTS` subqueries that alter aggregate scope or join cardinality. Every rejected pair carries a concrete witness database proving non-equivalence—100% of NEQ verdicts are confirmed.

(2) Complexity affects decidability. Breaking down by query complexity: on "simple" queries (single-table, few joins), the rejection rate is 79/645 (12.2%), driven by TPC-DS's simple-but-tricky aggregation patterns. On "complex" queries (CTEs + subqueries, ≥7 joins), only 23/3,281 decided pairs are rejected (0.7%); however, the decided rate drops to 31.1% (vs. 85.3% for simple) due to solver timeouts on larger formulas.

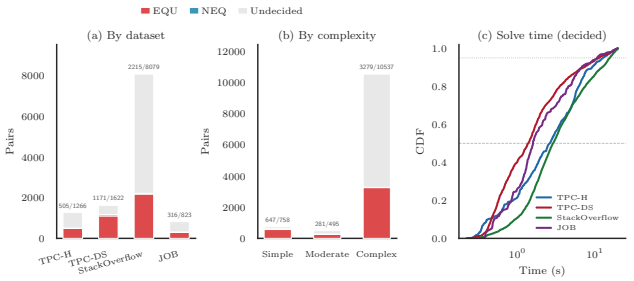

Figure 4: SQLStorm LLM-rewrite evaluation (Option 2). (a) Decision counts by dataset. (b) By query complexity. (c) Solve-time CDF for decided pairs (log scale). Numbers above bars show decided/total.

(3) Contrast with rule-based rewrites. The 8-vs-131 NEQ contrast between Options 1 and 2 validates CertOpt's role as a guardrail: rule-based engines like SQLStorm's compatibility pass almost never introduce semantic bugs (0.1% rejection rate), whereas LLM-generated rewrites fail at ≈28× higher rate (3.1%). This underscores the need for formal verification when deploying LLM-assisted query optimization in production.

Timing. Option 1 completes in 1,490 min (mean 7.0 s/pair, median 2.8 s); Option 2 in 1,328 min (mean 6.8 s/pair, median 2.9 s). Figure 4 shows the per-dataset decision rates and solve-time distributions.

8.5 Ablation Studies

We isolate the contribution of six design choices: the instance bound k, integrity constraints, witness validation, preprocessing, compositional decomposition, and family pruning. All ablations use validated witnesses and k=3 unless otherwise noted.

A1: Instance bound k. Table 8 sweeps k from 1 to 5 across three VeriEQL suites. Calcite and LeetCode peak at k=3 (89.7% and 83.3% decided) and degrade at $k \geq 4$: the larger SMT formulas push more pairs past the solver timeout, reducing the EQU count faster than NEQ gains compensate. Literature improves monotonically through k=5 (78.1%) because its schemas average only 3.6 tables, keeping formulas tractable even at higher bounds. The cost of higher k is substantial: mean solve time on LeetCode grows from 43 ms (k=1) to 7,020 ms (k=5), a 163× slowdown. The takeaway is that optimal k depends on *schema* complexity (table count and constraint density), not query complexity. For mixed workloads, k=3 provides the best accuracy–cost tradeoff.

A2: Integrity constraints. Table 9 shows the effect of removing PK/FK/`NOT NULL` constraints from the schema encoding. Without constraints, total false rejections explode from 7 to 236—a **33.7**× increase. On LeetCode alone, the false rejection rate jumps from 1.2% to 36.2%: the solver fabricates witness databases that violate integrity constraints (e.g., duplicate primary keys, dangling foreign keys) and therefore cannot exist in any valid instance. This confirms that faithful constraint encoding is the single most important design decision for soundness.

Table 8: Decision rate and timing vs. instance bound k. Decision rate = (EQU + NEQ) / total. Bold: best decision rate per suite.

	Decision rate (%)			LeetCode-1K	
k	Calcite	Lit.	LC-1K	Mean (ms)	p_{95} (ms)
1	87.2	59.4	79.8	43	81
2	87.7	64.1	80.6	888	502
3	**89.7**	75.0	**83.3**	2,410	30,087
4	89.2	76.6	80.8	4,511	30,400
5	86.4	**78.1**	78.9	7,020	34,486

Table 9: Safety ablation: false rejections (our NEQ where VeriEQL proves EQU) under three configurations at k=3. −Val: witness validation disabled. −IC: integrity constraints stripped.

Config	Calcite	Lit.	LC-1K	**Total**	LC FR %
Full (baseline)	0	0	7	**7**	1.2
−Validation	0	0	6	6	1.0
−Constraints	18	2	216	**236**	36.2

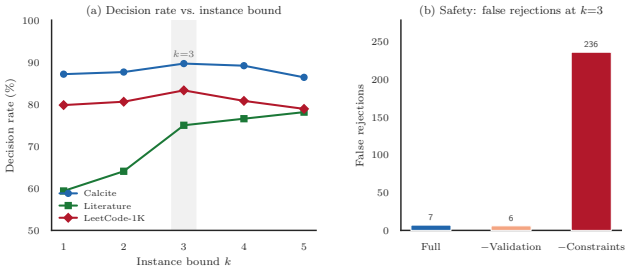

Figure 5: Ablation studies. (a) Decision rate vs. instance bound k for three VeriEQL suites. Calcite and LeetCode peak at k=3; Literature improves monotonically. (b) False rejections under three configurations at k=3. Removing integrity constraints causes a 33.7× increase.

A3: Witness validation. Disabling witness validation reduces total false rejections from 7 to 6: validation catches exactly one spurious SAT witness across all three suites. While the marginal impact is small, validation serves as a defense-in-depth layer that detects solver over-approximation at negligible cost (a single SQLite `SELECT` per SAT result).

A4: Preprocessing. Predicate promotion and table elimination have negligible impact on pairwise VeriEQL checks (Calcite: 89.7% decided with and without preprocessing). However, preprocessing is *essential* for the optimizer loop: on JOB-Complex without preprocessing, the solver failed to verify any of the first 5 queries in 73 min, compared to the baseline of 27/30 verified in 8 min. Preprocessing normalizes join structure before the solver sees it, reducing the effective number of terms in the equivalence formula.

Table 10: Compositional vs. monolithic verification on JOB-Complex (30 queries). Compositional decomposition recovers full verification at $k \geq 4$ where monolithic collapses.

Strategy	k=3	k=4	k=5
Monolithic	27/30 (492 s)	0/30 (4 s)	0/30 (4 s)
Compositional	27/30 (497 s)	**27/30** (4,467 s)	**27/30** (25,483 s)

A5: Compositional vs. monolithic on JOB-Complex. Table 10 compares monolithic and compositional verification on JOB-Complex across k=3–5. At k=3 the two strategies are equivalent (27/30 in $\approx$ 500 s). At $k \geq 4$, however, the monolithic solver collapses entirely: **0/30** queries verified, because the SMT formula for 9–16-table joins at k=4 exceeds the solver's capacity and every candidate returns Unknown. Compositional per-join decomposition (§6) recovers **27/30** at both k=4 (4,467 s) and k=5 (25,483 s) by breaking the global formula into tractable per-join subproblems. The same 3 queries (C05, C09, C20) fail under all configurations; they involve correlated predicates spanning 12+ joins that cannot be cleanly decomposed.

A6: Family pruning. On the BIRD benchmark [25] (500 queries, 3,071 candidates, mean 6.1 per query), family pruning reduces solver invocations by 33.1% (1,015 calls saved) and solver time by 25.6% (201.8 s). Full results are in Appendix O.

9 RELATED WORK

SQL equivalence verification. Cosette [9] pioneered automated SQL equivalence proving by encoding queries over K-relations into Coq-checkable proof obligations. Chu et al. [8] subsequently developed an axiomatic framework grounded in U-semiring theory, while HoTTSQL [10] leveraged Homotopy Type Theory for mechanized proofs of relational rewrites. SPES [47] introduced practical SMT-based verification under set semantics, and SQLSolver [15] decided equivalence via reduction to linear integer arithmetic. VeriEQL [19] represents the current state of the art: a bounded model checker that incorporates integrity constraints and proves equivalence on over 15,200 of 24,455 query pairs from Calcite, LeetCode, and the research literature. SpotIt [22] applies SMT-based bounded verification to evaluate text-to-SQL datasets, and SpotIt+ [23] extends it with mined integrity constraints atop VeriEQL. Polygon [45] introduces conflict-driven under-approximation for deciding equivalence, and QED [39] combines multiple decision procedures to handle complex SQL features. Most recently, QO-Verify [31] repurposes the optimizer's own memo structure to verify LLM-generated rewrites, achieving 37% coverage on 3,100+ candidates; however, it cannot *disprove* equivalence and explicitly identifies SMT-based augmentation as an open direction. Notably, QO-Verify reports—corroborating findings from LITHE [14]—that state-of-the-art SMT-based equivalence checkers could not verify any TPC-DS LLM rewrites; in contrast, CertOpt decides 1,171 of 1,622 TPC-DS LLM rewrites (§8): 1,103 verified equivalent, 68 *disproved* with concrete witness databases, and 451 undecided (timeout or unknown)—a 72.2% decision rate versus QO-Verify's 35% on TPC-DS, with a median verification time of 1.3 s and p95 of 20.0 s, on the same benchmark family that QO-Verify deemed out of reach for SMT-based methods. Guagliardo and Libkin [18] provide a formal semantics of SQL queries including NULL handling that informs our three-valued encoding, and Mohamed et al. [30] propose an SMT theory of tables and relations within cvc5 but do not yet support aggregation. All of these systems operate as *standalone* verification or evaluation tools; none is integrated into an optimization loop that generates, certifies, and ranks rewrites end-to-end.

Query optimization: traditional and learned. Traditional optimizers such as Volcano/Cascades [17] and Apache Calcite [6] apply algebraic rules without per-transformation semantic proofs; correctness relies on manual rule auditing and regression testing. Learned optimizers—Neo [29], Bao [28], Balsa [43], Lero [48]—improve plan selection via reinforcement learning or learning-to-rank, but operate at the physical plan level and do not certify that a rewritten SQL query preserves semantics. AutoSteer [2] steers optimizers via learned hint sets, and LLMSteer [1] uses LLM embeddings for hint-set classification (71% accuracy); neither verifies the semantic correctness of the resulting plans. LLM-R^2 [26] demonstrates that unconstrained LLM rewrites fail 92% of the time on TPC-H, and addresses this by constraining the LLM to select from pre-verified Calcite rules; CertOpt tackles the same risk from the verification side, certifying rewrites from *any* source—including unconstrained LLM outputs—without restricting the rewrite space. LITHE [14] combines LLM-proposed rewrites with MCTS exploration, but could formally verify only 40.7% of its cost-productive rewrites using QED/SQLSolver; the remaining 59.3% rely on statistical sampling, a gap that CertOpt's bounded verification is designed to fill. CertiQL [7] and Q*cert [3] provide machine-checked correctness for query *compilation* pipelines, but neither addresses the optimizer rewrite loop where candidates must be verified and ranked at query time. CertOpt is complementary: it certifies that SQL-level rewrites preserve bag semantics, regardless of whether candidates originate from rules, learned models, or large language models.

CEGIS and synthesis in databases. Counterexample-Guided Inductive Synthesis (CEGIS), formalized by Solar-Lezama [36] and inspired by the counterexample-guided abstraction refinement (CEGAR) paradigm of Clarke et al. [11], has been widely adopted in program synthesis. Within databases, synthesis techniques have been used to *construct* queries from natural language or input–output examples—SQLizer [41], SQUARES [32], Scythe [38], and Morpheus [16]—but these systems synthesize a query that satisfies a user-provided specification, rather than certifying an optimizer-proposed rewrite against a formal equivalence criterion. The Oracle-Guided Inductive Synthesis (OGIS) theory of Jha and Seshia [21] provides termination guarantees when the candidate space is finite, and Jha and Seshia [20] show that minimal counterexamples do not expand the class of solvable synthesis problems but can improve fault localization. Our work adapts CEGIS to a fundamentally different role: rather than synthesizing a program from scratch, we use the CEGIS loop as a *correctness filter* inside query optimization, where the "specification" is the original query, "candidates" are proposed rewrites, and "counterexamples" are distinguishing witness databases.

Database testing. SQLancer [33, 34, 46], developed by Rigger and collaborators, has discovered hundreds of logic bugs across more than twenty production DBMSs through complementary

techniques: Pivoted Query Synthesis (PQS), Ternary Logic Partitioning (TLP), and Non-optimizing Reference Engine Construction (NoREC). Ba and Rigger extend this line with QPG [4] (53 bugs via query plan guidance) and DQP [5] (26 bugs via differential query plans), while CODDTest [44] finds 24 logic bugs through constant-optimization-driven testing. QEX [37] generates symbolic test databases via constraint solving to improve query-level coverage. SQLDriller [42] synthesizes counterexample databases via VeriEQL/SQLSolver to validate and fix text-to-SQL dataset mappings. These testing approaches motivate witness-grounded rejection: if a concrete database distinguishes two queries, they are definitively inequivalent. CertOpt extends this insight from post-hoc testing to online verification within an optimization loop, materializing witnesses in SQLite to validate SMT-produced counterexamples and conservatively returning Unknown when the solver is inconclusive. To our knowledge, CertOpt is the first system to close the *generate* → *verify* → *certify* loop for query optimization, combining all four concerns—equivalence verification, rewrite generation, witness-grounded rejection, and proof-carrying output—in a single pipeline.

10 CONCLUSION

We presented CertOpt, a proof-carrying query optimization framework that closes the generate–verify–certify loop via CEGIS-based bounded equivalence checking. On 24,455 VeriEQL pairs CertOpt achieves zero false equivalence claims with up to 582× speedup; on 24,495 SQLStorm pairs it catches 131 incorrect LLM rewrites (3.1% rejection rate); and compositional decomposition scales verification to 27/30 JOB-Complex queries with up to 16 tables.

Limitations and future work. Σ-equivalence is a safety net, not a completeness guarantee. Recursive CTEs, `LATERAL` joins, and non-literal `LIKE` patterns receive conservative Unknown verdicts. Future extensions include outer-join composition, Z3 string theory for `LIKE`, and learned cost models.

REFERENCES

[1] Peter Akioyamen, Zixuan Yi, and Ryan Marcus. 2024. The Unreasonable Effectiveness of LLMs for Query Optimization. arXiv:2411.02862 [cs.DB] https://arxiv.org/abs/2411.02862

[2] Christoph Anneser, Nesime Tatbul, David Cohen, Zhenggang Xu, Prithviraj Pandian, Nikolay Laptev, and Ryan Marcus. 2023. AutoSteer: Learned Query Optimization for Any SQL Database. *Proc. VLDB Endow.* 16, 12 (Aug. 2023), 3515–3527. https://doi.org/10.14778/3611540.3611544

[3] Joshua S. Auerbach, Martin Hirzel, Louis Mandel, Avraham Shinnar, and Jérôme Siméon. 2017. Handling Environments in a Nested Relational Algebra with Combinators and an Implementation in a Verified Query Compiler. In *Proceedings of the 2017 ACM International Conference on Management of Data* (Chicago, Illinois, USA) *(SIGMOD '17)*. Association for Computing Machinery, New York, NY, USA, 1555–1569. https://doi.org/10.1145/3035918.3035961

[4] Jinsheng Ba and Manuel Rigger. 2023. Testing Database Engines via Query Plan Guidance. In *Proceedings of the 45th International Conference on Software Engineering* (Melbourne, Victoria, Australia) *(ICSE '23)*. IEEE Press, 2060–2071. https://doi.org/10.1109/ICSE48619.2023.00174

[5] Jinsheng Ba and Manuel Rigger. 2024. Keep It Simple: Testing Databases via Differential Query Plans. *Proc. ACM Manag. Data* 2, 3, Article 188 (May 2024), 26 pages. https://doi.org/10.1145/3654991

[6] Edmon Begoli, Jesús Camacho-Rodríguez, Julian Hyde, Michael J. Mior, and Daniel Lemire. 2018. Apache Calcite: A Foundational Framework for Optimized Query Processing Over Heterogeneous Data Sources. In *Proceedings of the 2018 International Conference on Management of Data* (Houston, TX, USA) *(SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 221–230. https://doi.org/10.1145/3183713.3190662

[7] Véronique Benzaken, Évelyne Contejean, and Stefania Dumbrava. 2014. A Coq Formalization of the Relational Data Model. In *Proceedings of the 23rd European Symposium on Programming Languages and Systems - Volume 8410*. Springer-Verlag, Berlin, Heidelberg, 189–208. https://doi.org/10.1007/978-3-642-54833-8_11

[8] Shumo Chu, Brendan Murphy, Jared Roesch, Alvin Cheung, and Dan Suciu. 2018. Axiomatic foundations and algorithms for deciding semantic equivalences of SQL queries. *Proc. VLDB Endow.* 11, 11 (July 2018), 1482–1495. https://doi.org/10.14778/3236187.3236200

[9] Shumo Chu, Chenglong Wang, Konstantin Weitz, and Alvin Cheung. 2017. Cosette: An Automated Prover for SQL. In *CIDR*.

[10] Shumo Chu, Konstantin Weitz, Alvin Cheung, and Dan Suciu. 2017. HoTTSQL: proving query rewrites with univalent SQL semantics. *SIGPLAN Not.* 52, 6 (June 2017), 510–524. https://doi.org/10.1145/3140587.3062348

[11] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. 2003. Counterexample-guided abstraction refinement for symbolic model checking. *J. ACM* 50, 5 (Sept. 2003), 752–794. https://doi.org/10.1145/876638.876643

[12] Samuel Colvin. 2026. Pydantic: Data Validation for Python. https://pydantic.dev/

[13] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: an efficient SMT solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems* (Budapest, Hungary) *(TACAS'08/ETAPS'08)*. Springer-Verlag, Berlin, Heidelberg, 337–340.

[14] Sriram Dharwada, Himanshu Devrani, Jayant R. Haritsa, and Harish Doraiswamy. 2026. LITHE: A Query Rewrite Advisor using LLMs. In *Proceedings 29th International Conference on Extending Database Technology, EDBT 2026, Tampere, Finland, March 24-27, 2026*, Wolfgang Lehner, Vanessa Braganholo, Kostas Stefanidis, Zheying Zhang, Alexander Krause, and João Felipe Nicolaci Pimentel (Eds.). OpenProceedings.org, 233–246. https://doi.org/10.48786/EDBT.2026.20

[15] Haoran Ding, Zhaoguo Wang, Yicun Yang, Dexin Zhang, Zhenglin Xu, Haibo Chen, Ruzica Piskac, and Jinyang Li. 2023. Proving Query Equivalence Using Linear Integer Arithmetic. *Proc. ACM Manag. Data* 1, 4, Article 227 (Dec. 2023), 26 pages. https://doi.org/10.1145/3626768

[16] Yu Feng, Ruben Martins, Jacob Van Geffen, Isil Dillig, and Swarat Chaudhuri. 2017. Component-based synthesis of table consolidation and transformation tasks from examples. *SIGPLAN Not.* 52, 6 (June 2017), 422–436. https://doi.org/10.1145/3140587.3062351

[17] Goetz Graefe and William J. McKenna. 1993. The Volcano Optimizer Generator: Extensibility and Efficient Search. In *Proceedings of the Ninth International Conference on Data Engineering*. IEEE Computer Society, USA, 209–218.

[18] Paolo Guagliardo and Leonid Libkin. 2017. A formal semantics of SQL queries, its validation, and applications. *Proc. VLDB Endow.* 11, 1 (Sept. 2017), 27–39. https://doi.org/10.14778/3151113.3151116

[19] Yang He, Pinhan Zhao, Xinyu Wang, and Yuepeng Wang. 2024. VeriEQL: Bounded Equivalence Verification for Complex SQL Queries with Integrity Constraints. *Proc. ACM Program. Lang.* 8, OOPSLA1, Article 132 (April 2024), 29 pages. https://doi.org/10.1145/3649849

[20] Susmit Jha and Sanjit A. Seshia. 2014. Are There Good Mistakes? A Theoretical Analysis of CEGIS. In *SYNT (EPTCS 157)*. 84–99. https://doi.org/10.4204/EPTCS.157.10

[21] Susmit Jha and Sanjit A. Seshia. 2017. A theory of formal synthesis via inductive learning. *Acta Inf.* 54, 7 (Nov. 2017), 693–726. https://doi.org/10.1007/s00236-017-0294-5

[22] Rocky Klopfenstein, Yang He, Andrew Tremante, Yuepeng Wang, Nina Narodytska, and Haoze Wu. 2026. SpotIt: Evaluating Text-to-SQL Evaluation with Formal Verification. arXiv:2510.26840 [cs.DB] https://arxiv.org/abs/2510.26840

[23] Rocky Klopfenstein, Yang He, Andrew Tremante, Yuepeng Wang, Nina Narodytska, and Haoze Wu. 2026. SpotIt+: Verification-based Text-to-SQL Evaluation with Database Constraints. arXiv:2603.04334 [cs.DB] https://arxiv.org/abs/2603.04334

[24] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2025. Still Asking: How Good Are Query Optimizers, Really? *Proc. VLDB Endow.* 18, 12 (Aug. 2025), 5531–5536. https://doi.org/10.14778/3750601.3760521

[25] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C.C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM already serve as a database interface? a big bench for large-scale database grounded text-to-SQLs. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 1835, 28 pages.

[26] Zhaodonghui Li, Haitao Yuan, Huiming Wang, Gao Cong, and Lidong Bing. 2024. LLM-R2: A Large Language Model Enhanced Rule-Based Rewrite System for Boosting Query Efficiency. *Proc. VLDB Endow.* 18, 1 (Sept. 2024), 53–65. https://doi.org/10.14778/3696435.3696440

[27] Toby Mao. 2026. sqlglot: SQL parser and transpiler. https://github.com/tobymao/sqlglot

[28] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making Learned Query Optimization Practical. In *Proceedings of the 2021 International Conference on Management of Data* (Virtual Event, China) *(SIGMOD '21)*. Association for Computing Machinery, New

York, NY, USA, 1275–1288. https://doi.org/10.1145/3448016.3452838
[29] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Chi Zhang, Mohammad Alizadeh, Tim Kraska, Olga Papaemmanouil, and Nesime Tatbul. 2019. Neo: a learned query optimizer. *Proc. VLDB Endow.* 12, 11 (July 2019), 1705–1718. https://doi.org/10.14778/3342263.3342644
[30] Mudathir Mahgoub Yahia Mohamed, Andrew Reynolds, Cesare Tinelli, and Clark Barrett. 2024. Verifying SQL queries using theories of tables and relations. In *Proceedings of 25th Conference on Logic for Programming, Artificial Intelligence and Reasoning (EPiC Series in Computing)*, Nikolaj Bjørner, Marijn Heule, and Andrei Voronkov (Eds.), Vol. 100. EasyChair, 445–463. https://doi.org/10.29007/rlt7
[31] Vivek R. Narasayya and Surajit Chaudhuri. 2026. Leveraging Query Optimizers to Verify the Soundness of LLM-based Query Rewrites for Real-World Workloads, and More!. In *16th Conference on Innovative Data Systems Research, CIDR 2026, Chaminade, CA, USA, January 18-21, 2026*. www.cidrdb.org. https://vldb.org/cidrdb/2026/leveraging-query-optimizers-to-verify-the-soundness-of-llm-based-query-rewrites-for-real-world-workloads.html
[32] Pedro Orvalho, Miguel Terra-Neves, Miguel Ventura, Ruben Martins, and Vasco Manquinho. 2020. SQUARES: a SQL synthesizer using query reverse engineering. *Proc. VLDB Endow.* 13, 12 (Aug. 2020), 2853–2856. https://doi.org/10.14778/3415478.3415492
[33] Manuel Rigger and Zhendong Su. 2020. Finding bugs in database systems via query partitioning. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 211 (Nov. 2020), 30 pages. https://doi.org/10.1145/3428279
[34] Manuel Rigger and Zhendong Su. 2020. Testing database engines via pivoted query synthesis. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation (OSDI'20)*. USENIX Association, USA, Article 38, 16 pages.
[35] Tobias Schmidt, Viktor Leis, Peter Boncz, and Thomas Neumann. 2025. SQLStorm: Taking Database Benchmarking into the LLM Era. *Proc. VLDB Endowment* 18, 11 (2025).
[36] Armando Solar-Lezama. 2013. Program sketching. *Int. J. Softw. Tools Technol. Transf.* 15, 5–6 (Oct. 2013), 475–495. https://doi.org/10.1007/s10009-012-0249-7
[37] Margus Veanes, Pavel Grigorenko, Peli Halleux, and Nikolai Tillmann. 2009. Symbolic Query Exploration. In *Proceedings of the 11th International Conference on Formal Engineering Methods: Formal Methods and Software Engineering* (Rio de Janeiro, Brazil) *(ICFEM '09)*. Springer-Verlag, Berlin, Heidelberg, 49–68. https://doi.org/10.1007/978-3-642-10373-5_3
[38] Chenglong Wang, Alvin Cheung, and Rastislav Bodik. 2017. Synthesizing highly expressive SQL queries from input-output examples. *SIGPLAN Not.* 52, 6 (June 2017), 452–466. https://doi.org/10.1145/3140587.3062365
[39] Shuxian Wang, Sicheng Pan, and Alvin Cheung. 2024. QED: A Powerful Query Equivalence Decider for Complex SQL. *Proc. VLDB Endow.* 17, 11 (July 2024), 3602–3614. https://doi.org/10.14778/3681954.3682024
[40] Johannes Wehrstein, Timo Eckmann, Roman Heinrich, and Carsten Binnig. 2025. JOB-Complex: A Challenging Benchmark for Traditional & Learned Query Optimization. arXiv:2507.07471 [cs.DB] https://arxiv.org/abs/2507.07471
[41] Navid Yaghmazadeh, Yuepeng Wang, Isil Dillig, and Thomas Dillig. 2017. SQLizer: query synthesis from natural language. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 63 (Oct. 2017), 26 pages. https://doi.org/10.1145/3133887
[42] Yicun Yang, Zhaoguo Wang, Yu Xia, Zhuoran Wei, Haoran Ding, Ruzica Piskac, Haibo Chen, and Jinyang Li. 2025. Automated Validating and Fixing of Text-to-SQL Translation with Execution Consistency. *Proc. ACM Manag. Data* 3, 3, Article 134 (June 2025), 28 pages. https://doi.org/10.1145/3725271
[43] Zongheng Yang, Wei-Lin Chiang, Sifei Luan, Gautam Mittal, Michael Luo, and Ion Stoica. 2022. Balsa: Learning a Query Optimizer Without Expert Demonstrations. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 931–944. https://doi.org/10.1145/3514221.3517885
[44] Chi Zhang and Manuel Rigger. 2025. Constant Optimization Driven Database System Testing. *Proc. ACM Manag. Data* 3, 1, Article 24 (Feb. 2025), 24 pages. https://doi.org/10.1145/3709674
[45] Pinhan Zhao, Yuepeng Wang, and Xinyu Wang. 2025. Polygon: Symbolic Reasoning for SQL using Conflict-Driven Under-Approximation Search. *Proc. ACM Program. Lang.* 9, PLDI, Article 200 (June 2025), 26 pages. https://doi.org/10.1145/3729303
[46] Suyang Zhong and Manuel Rigger. 2026. Scaling Automated Database System Testing. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (USA) *(ASPLOS '26)*. Association for Computing Machinery, New York, NY, USA, 1677–1692. https://doi.org/10.1145/3779212.3790215
[47] Qi Zhou, Joy Arulraj, Shamkant Navathe, William Harris, and Dong Xu. 2019. Automated verification of query equivalence using satisfiability modulo theories. *Proc. VLDB Endow.* 12, 11 (July 2019), 1276–1288. https://doi.org/10.14778/3342263.3342267
[48] Rong Zhu, Wei Chen, Bolin Ding, Xingguang Chen, Andreas Pfadler, Ziniu Wu, and Jingren Zhou. 2023. Lero: A Learning-to-Rank Query Optimizer. *Proc. VLDB Endow.* 16, 6 (Feb. 2023), 1466–1479. https://doi.org/10.14778/3583140.3583160

A FULL PROOF DETAILS

This appendix provides complete proofs for the theorems stated in §7 and §6. We first establish a chain of supporting lemmas (Appendix A.1) that formalize the correspondence between the symbolic encoding and SQL semantics, then use them to prove the main theorems.

A.1 Supporting Lemmas

We write $\hat{D}$ for the symbolic database (a set of Z3 variables representing k rows per table), ρ for a satisfying assignment (a concrete valuation of all Z3 variables), and D_ρ for the concrete database instance induced by ρ. We write $\mathcal{F}_{\text{exact}}$ for the *exact fragment*: the set of SQL constructs for which the encoding is faithful (all constructs in Table 11 marked "Exact").

Notation and conventions. Each symbolic row carries a Boolean *present* flag $present[T][i]$ that indicates whether row i of table T exists in the concrete instance. At each cardinality k, the solver allocates k symbolic rows per table; rows with $present = \bot$ represent absent rows and are excluded from query evaluation. This mechanism allows a single call at cardinality k to model all instances with *at most k* rows per table, including 0-row tables and mixed-cardinality instances.

We write $\phi_{\mathcal{D}}$ for the conjunction of all domain and integrity constraints on the symbolic database (value bounds, integrality, PK/UNIQUE distinctness, FK referencing, NOT NULL flags, and *present*-flag consistency). We write $\hat{e}(\rho, \eta)$ for the symbolic evaluation of a scalar expression e and $\hat{p}(\rho, \eta)$ for the symbolic evaluation of a predicate p, both under assignment ρ and *combo environment* η—a mapping from table aliases to symbolic row indices. The faithful-encoding claims below apply to $\hat{e}$ and $\hat{p}$ respectively. Lemmas A.2 and A.3 are proved by *simultaneous structural induction*: CASE WHEN branch conditions (predicates inside expressions) and aggregate arguments (expressions inside aggregates) create mutual dependencies between the expression and predicate strata. The induction is well-founded because each recursive call descends in AST height.

Lemma A.1 (Symbolic Instance Realization). *At each cardinality k, there is a surjection from satisfying assignments ρ of the symbolic database variables (subject to domain constraints $\phi_{\mathcal{D}}$ and the present flags) onto concrete database instances with at most k rows per table in $\mathsf{Inst}(\Sigma, \mathcal{S})$.*

Specifically:

(1) (Completeness.) *For every $D \in \mathsf{Inst}(\Sigma, \mathcal{S})$ with at most k rows per table, there exists an assignment ρ_D such that $\rho_D \models \phi_{\mathcal{D}}$ and $D_{\rho_D} = D$.*
(2) (Soundness.) *For every ρ with $\rho \models \phi_{\mathcal{D}}$, the induced instance $D_\rho \in \mathsf{Inst}(\Sigma, \mathcal{S})$.*

Proof. Each table T is allocated k symbolic rows, each carrying a Boolean $present[T][i]$ flag and nullable cells per column. A row with $present = \bot$ is *absent*: it does not contribute to query evaluation, allowing a single solver call at cardinality k to represent instances where individual tables have anywhere from 0 to k rows (including mixed-cardinality instances such as $|R|$=1, $|S|$=3 when k≥3).

The domain constraints $\phi_{\mathcal{D}}$ enforce: (a) value bounds matching $\mathcal{D}$, (b) integrality for integer columns, (c) PK/UNIQUE distinctness among present rows, (d) FK referencing among present rows, and (e) NOT NULL flags for present rows. Absent rows are unconstrained in their value fields (their *val* components are don't-cares), so the mapping from assignments to instances is a surjection rather than a strict bijection. These constraints are exactly the defining conditions of $\mathsf{Inst}(\Sigma, \mathcal{S})$ (Definition 2.4) restricted to present rows.

For *completeness*: given D with $|T| = n_T \le k$ for each table T, set $present[T][i] = \top$ for $i < n_T$ and $\bot$ otherwise. For present rows, set each cell's *is_null* to the cell's NULL status and *val* to the encoded value (integer rank for strings, direct value for numerics). The domain constraints hold by construction since $D \in \mathsf{Inst}(\Sigma, \mathcal{S})$.

For *soundness*: given $\rho \models \phi_{\mathcal{D}}$, restrict to present rows and decode each cell to recover a concrete value. The domain constraints ensure all integrity constraints hold among present rows, so $D_\rho \in \mathsf{Inst}(\Sigma, \mathcal{S})$.

The at-most-K ascending dense schedule (§2.3) runs the solver at each $k = 1, \ldots, K$. Since a single call at cardinality k already covers all instances with at most k rows per table (via the *present* flags), the schedule at k=K alone suffices to cover $\mathsf{Inst}(\Sigma, \mathcal{S})$. Running the full schedule $k = 1, \ldots, K$ provides incremental results and early termination without affecting coverage. □

Lemma A.2 (Expression Evaluation Soundness). *For every* row-local *scalar expression e built from* `ColumnRef`, `Literal`, `BinOp` *(+, −, ×, /),* `COALESCE`, `CASE WHEN`, `IS NULL`, `NULLIF`, `ABS`, `CAST`, *and exact* `FuncCall` *nodes (Table 11), every satisfying assignment ρ, and every combo environment η, the symbolic evaluation $\hat{e}(\rho, \eta)$ equals the SQL evaluation $[\![e]\!]_{D_\rho, \eta}$ as a nullable value:*

$$\hat{e}(\rho, \eta).is\_null = \top \iff [\![e]\!]_{D_\rho, \eta} = \mathsf{NULL}$$

and when both are non-null, $\hat{e}(\rho, \eta).val = [\![e]\!]_{D_\rho, \eta}$.

Aggregate expressions (`AggCall`*) and window functions (*`WindowFunc`*) are proved separately at a higher stratum: see Lemma A.5 and Appendix H respectively.*

Proof. By simultaneous structural induction on expressions and predicates (jointly with Lemma A.3), descending in AST height.

Base cases. (i) `ColumnRef`: the symbolic cell for the present row (identified by η) directly encodes the column value, so the result follows from Lemma A.1. (ii) `Literal`: a constant is encoded as a Z3 constant of the same value; the correspondence is immediate.

Inductive cases. (iii) `BinOp` (+, −, ×, /): the encoding propagates NULLs (if either operand is null, the result is null) and applies the Z3 arithmetic operator otherwise. SQL specifies the same NULL propagation and arithmetic, so the inductive hypothesis on both operands yields the result. Division uses Real sort, matching SQL's exact-division semantics. (iv) `COALESCE`$(e_1, \ldots, e_n)$: the encoding is a right-fold of $\mathsf{If}(\hat{e}_i.is\_null, \ldots)$, which selects the first non-null argument. By induction, each $\hat{e}_i$ is faithful, so the fold result is faithful. (v) `CASE WHEN`: each branch condition is a predicate; by the simultaneous induction hypothesis on Lemma A.3 (which applies at strictly smaller AST height), the branch conditions are faithful. The first definitely-true branch determines the result; by induction on the branch expressions, the encoding is faithful. (vi) `IS NULL`,

NULLIF, ABS, CAST: each is a compositional function on faithful sub-expressions with deterministic NULL propagation; the inductive hypothesis applies. □

Lemma A.3 (Three-Valued Predicate Soundness). *For every predicate p built from comparisons ($=, \neq, <, \leq, >, \geq$), Boolean connectives (And, Or, Not), IS NULL, IN lists, BETWEEN, and literal LIKE patterns, every satisfying assignment ρ, and every combo environment η, the encoded TriBool $\hat{p}(\rho, \eta) = \langle u, v \rangle$ faithfully represents the SQL three-valued evaluation:*

- $\hat{p}(\rho, \eta) = \langle \bot, \top \rangle$ *iff* $[\![p]\!]_{D_\rho, \eta} =$ True,
- $\hat{p}(\rho, \eta) = \langle \bot, \bot \rangle$ *iff* $[\![p]\!]_{D_\rho, \eta} =$ False,
- $\hat{p}(\rho, \eta) = \langle \top, * \rangle$ *iff* $[\![p]\!]_{D_\rho, \eta} =$ Unknown.

Subquery-form predicates (EXISTS, IN (subquery)) are handled by recursive query evaluation (§5.3) and inherit soundness from the query-level encoding; their correctness is covered by Lemma A.6 at the inner-query level.

Proof. By simultaneous structural induction with Lemma A.2, descending in AST height.

Base case (comparison). For e_1 *op* e_2 with *op* $\in \{=, \neq, <, \leq, >, \geq\}$: by Lemma A.2, $\hat{e}_1(\rho, \eta)$ and $\hat{e}_2(\rho, \eta)$ are faithful. The encoding sets *is_unknown* $= \hat{e}_1.is\_null \vee \hat{e}_2.is\_null$ and *val* to the comparison result. This matches SQL's rule: if either operand is NULL, the comparison yields Unknown; otherwise it yields a definite Boolean.

Inductive cases. (i) And(a, b): the encoding implements False-dominance: *is_unk* $= \neg(a_f \vee b_f) \wedge (a.unk \vee b.unk)$ where $a_f = \neg a.unk \wedge \neg a.val$. This is exactly Kleene's strong AND: False $\wedge$ Unknown = False, Unknown $\wedge$ True = Unknown. By induction on a and b, the result is faithful. (ii) Or(a, b): symmetric, with True-dominance. (iii) Not(a): unknown is preserved, val is negated, matching SQL's Not(Unknown) = Unknown. (iv) IsNull(e): returns $\langle \bot,\ e.is\_null \rangle$, which is always definite. This matches SQL. (v) IN list: expanded to an OR chain of equalities; faithful by cases (i) and the base case. (vi) BETWEEN: expanded to And$(\geq, \leq)$; faithful by (i) and the base case. □

Lemma A.4 (Join Evaluation Soundness). *For queries in $\mathcal{F}_{\text{exact}}$, the symbolic combo survival flags $\{survives[c]\}_{c \in [0,k^{|\mathcal{A}|})}$ faithfully encode SQL join semantics: for every satisfying assignment ρ, a combo $c = (i_1, \ldots, i_m)$ survives (i.e., $\rho \models survives[c]$) if and only if the corresponding row tuple survives the SQL join evaluation on D_ρ.*

Specifically:

(1) INNER JOIN: *$survives[c] \Leftrightarrow tb\_true(\hat{\theta}(c))$, where $\hat{\theta}$ is the symbolic ON predicate. By Lemma A.3, this holds iff the ON predicate is definitely True under SQL 3VL.*

(2) LEFT OUTER JOIN: *For each left-side row i, the encoding emits matched pairs where ON is True and a single NULL-padded row when no match exists. The NULL-padded row is represented as a synthetic SymbolicRow whose every column has is_null $= \top$ (via MakeNullRow), and whose present flag is set to the unmatched condition:*

$$unmatched[i] = \bigwedge_j \neg\, tb\_true(\hat{\theta}(i, j)).$$

This matches the SQL LEFT JOIN specification: matched pairs produce real combos; unmatched left rows produce exactly one NULL-extended combo.

(3) RIGHT and FULL OUTER JOIN: *Symmetric to LEFT (RIGHT) or union of both unmatched sides (FULL). Matched rows appear once; each unmatched row produces one NULL-padded combo.*

(4) CROSS JOIN: *All combos survive unconditionally.*

Multi-join composition: *Joins are processed left-to-right. At each step, the set of surviving combos from the previous joins forms the "left side" for the next join. This iterative construction mirrors SQL's nested-loop join semantics and preserves bag multiplicities because each combo is an independent row-index tuple.*

Proof. For INNER JOIN: a combo c survives iff $tb\_true(\hat{\theta}(c)) = \neg\hat{\theta}.unk \wedge \hat{\theta}.val$. By Lemma A.3, $\hat{\theta}(c)$ is faithful, so this condition holds iff the SQL ON predicate is True—exactly the SQL INNER JOIN semantics.

For LEFT OUTER JOIN: the matched case follows from INNER JOIN. For the unmatched case, $unmatched[i]$ is true iff no right-side row j makes $\hat{\theta}(i, j)$ True. By Lemma A.3, this holds iff the concrete ON predicate is never True for row i, which is exactly when SQL emits a NULL-padded row. The synthetic NULL-padded row is a SymbolicRow with all columns set to $\langle is\_null{=}\top, val{=}0 \rangle$; subsequent expressions correctly observe $is\_null = \top$ and propagate NULLs via Lemma A.2.

RIGHT and FULL OUTER JOIN are symmetric: RIGHT pads left-side aliases; FULL emits matched, unmatched-left, and unmatched-right combo sets. CROSS is trivial (no predicate).

Multi-join composition follows by induction on the number of joins: the survival flags at step j depend only on the surviving combos from step $j{-}1$ and the new right-side rows, matching SQL's left-to-right evaluation. □

Lemma A.5 (Aggregation Soundness). *For queries in $\mathcal{F}_{\text{exact}}$, the group representative encoding faithfully computes SQL aggregate semantics: for each group g in the SQL result, the corresponding group representative i in the symbolic encoding satisfies $\hat{f}(i, \rho) = [\![f]\!]_{g, D_\rho}$ for every aggregate function f in {Count, Count(*), Sum, Avg, Min, Max, Count Distinct}.*

Proof. *Group identity.* Two combos i, j belong to the same group iff $sg[i][j] = \top$ (Definition 5.4). The same-group predicate uses both-null-or-both-equal per GROUP BY key. By Lemma A.2, each key's symbolic value is faithful, so $sg[i][j]$ holds iff the concrete GROUP BY values agree under SQL equality (which treats NULLs as equal for grouping purposes, per the SQL standard).

Group representative uniqueness. $is\_rep[i]$ is true iff i survives and no earlier-indexed surviving combo belongs to the same group (Definition 5.5). Since the same-group relation is an equivalence relation on surviving combos, exactly one representative per group is selected (the minimum-indexed member). This establishes a bijection between symbolic group representatives and SQL groups.

Aggregate functions. For each representative i of group g:

- Count$(*) = \sum_j \text{If}(survives[j] \wedge sg[i][j], 1, 0)$. This counts all surviving combos in the group, matching SQL.
- Count$(e) = \sum_j \text{If}(survives[j] \wedge sg[i][j] \wedge \neg\hat{e}_j.null, 1, 0)$. By Lemma A.2, $\hat{e}_j.null$ is faithful, so this counts non-null values as SQL specifies.

- Sum(e): analogous, summing non-null $\hat{e}_j.val$ values. By Lemma A.2, each value is faithful, and the Z3 Real arithmetic is exact.
- Avg(e) = Sum(e)/Count(e) with exact Real division. Faithful by composition of the above.
- Min(e): the encoding uses guarded accumulation over the combo list, selecting the non-null value $\hat{e}_j.val$ such that no other non-null combo in the group has a strictly smaller value. By Lemma A.2 and the totality of the Real ordering, this selects the SQL minimum.
- Max(e): symmetric to MIN.
- Count Distinct(e): for each combo j in the group, j is counted only if it is the first non-null occurrence of its value. The first-occurrence check uses $\neg\exists j' < j : sg[i][j'] \wedge \hat{e}_j.val = \hat{e}_{j'}.val$. By Lemma A.2, value equality is faithful, so distinct values are correctly counted.

Empty groups: Count returns 0; all other aggregates return NULL (the encoding sets *is_null* = $\top$ when the group has no non-null values). This matches the SQL standard.

Scalar aggregate on empty input. When a query has aggregate functions but no `GROUP BY` clause and no input rows survive the WHERE filter, SQL specifies that exactly one result row is produced (Count returns 0; all other aggregates return NULL). The encoding handles this case via a synthetic result row whose survival condition is the negation of any input combo surviving. The synthetic row's column values are computed by EvalGroupValueEmpty: Count $\mapsto$ 0 (non-null), all other aggregates $\mapsto$ NULL. This ensures the symbolic result bag correctly contains one empty-aggregate row when the input is empty, matching the SQL standard. □

Lemma A.6 (Result-Row Soundness). *For queries in $\mathcal{F}_{\text{exact}}$ without ORDER BY + LIMIT (or whose ORDER BY keys fully determine the row order), the symbolic result bag $\hat{R}_Q$ faithfully represents the SQL result bag: for every satisfying assignment ρ, the multiset of concrete result rows induced by $\hat{R}_Q(\rho)$ equals $\{\!\{Q\}\!\}_{D_\rho}$.*

For queries with ORDER BY + LIMIT and potential ties, a weaker UNSAT safety *property holds: if $\hat{R}_{Q_1}$ and $\hat{R}_{Q_2}$ cannot be distinguished by the difference predicate, then no tie resolution on any bounded instance can produce differing result bags.*

Proof. *Non-aggregated queries.* Each result row is a projected tuple of a surviving combo. By Lemma A.4, the surviving combos faithfully represent the SQL join result. By Lemma A.2, the projected column values are faithful. If the query has a WHERE clause, the combo survives only when $tb\_true(\hat{p}_{where})$ holds; by Lemma A.3, this is faithful. Therefore $\hat{R}_Q(\rho) = \{\!\{Q\}\!\}_{D_\rho}$ as multisets.

Aggregated queries. By Lemma A.5, each group representative's aggregate values are faithful, and the set of representatives is in bijection with SQL groups. The HAVING filter (if present) is a TriBool predicate evaluated on aggregate values; by Lemma A.3, it is faithful. Surviving group representatives with their projected columns form $\hat{R}_Q(\rho) = \{\!\{Q\}\!\}_{D_\rho}$.

DISTINCT. The encoding caps each tuple's multiplicity at 1 via a first-occurrence check. This is equivalent to SQL's `SELECT DISTINCT` deduplication.

ORDER BY + LIMIT. The rank-based encoding (Appendix D) computes $rank(r)$ using $\prec_{ord}$ with existential tie-breakers. A result row r survives iff $rank(r) \le \ell$. Because each tie-breaker variable is unconstrained, the UNSAT verdict holds for *all* possible tie resolutions simultaneously: if the difference predicate is UNSAT, then no assignment of tie-breakers (combined with any database instance) can produce differing result bags. Note that SQL's tie behavior is underspecified; the existential encoding treats equivalence as "equal under all possible tie resolutions," which is the appropriate notion for UNSAT safety. In the SAT direction, the solver may find a witness that exploits a particular tie resolution; such witnesses are confirmed by the execution-based validation step (§7.2). □

Lemma A.7 (Difference Predicate Soundness). *For queries in $\mathcal{F}_{\text{exact}}$ whose result semantics is deterministic (no ORDER BY + LIMIT, or equivalently, the ORDER BY keys fully determine the row order), the difference predicate $\Delta(\hat{R}_{Q_1}, \hat{R}_{Q_2})$ (Definition 5.6) is satisfiable under $\phi_{\mathcal{D}}$ if and only if there exists $D \in \mathsf{Inst}(\Sigma, \mathcal{S})$ with $\{\!\{Q_1\}\!\}_D \neq \{\!\{Q_2\}\!\}_D$.*

For queries with ORDER BY + LIMIT and potential ties, the UNSAT safety *direction still holds: if Δ is unsatisfiable, then no instance distinguishes Q_1 from Q_2 under any tie resolution. In the SAT direction, the witness may exploit a specific tie resolution.*

Proof. *Deterministic case (no ORDER BY + LIMIT ties).*

($\Leftarrow$) Suppose D^* distinguishes Q_1 and Q_2: some tuple t has $mult(t, \{\!\{Q_1\}\!\}_{D^*}) \neq mult(t, \{\!\{Q_2\}\!\}_{D^*})$. By Lemma A.1, D^* corresponds to assignment ρ^* with $\rho^* \models \phi_{\mathcal{D}}$. By Lemma A.6 (deterministic case), $\hat{R}_{Q_1}(\rho^*)$ and $\hat{R}_{Q_2}(\rho^*)$ faithfully represent the result bags. Therefore there exists a symbolic result row r whose *count* differs between the two bags, making Δ satisfied.

($\Rightarrow$) Suppose ρ^* satisfies $\phi_{\mathcal{D}} \wedge \Delta$. By Lemma A.1, $D_{\rho^*} \in \mathsf{Inst}(\Sigma, \mathcal{S})$. By Lemma A.6 (deterministic case), the symbolic result bags are faithful, so the multiplicity difference witnessed by Δ corresponds to an actual difference in $\{\!\{Q_1\}\!\}_{D_{\rho^*}}$ vs. $\{\!\{Q_2\}\!\}_{D_{\rho^*}}$.

ORDER BY + LIMIT with ties (UNSAT safety only). When the queries use ORDER BY + LIMIT with potential ties, the existential tie-breaker encoding quantifies over all possible tie resolutions. If Δ is unsatisfiable, then *no* combination of database instance and tie resolution can produce differing result bags, establishing the UNSAT safety direction. The ($\Leftarrow$) direction also holds: a distinguishing instance with a specific tie resolution corresponds to a satisfying assignment where the tie-breaker variables are set to match that resolution. □

A.2 Proof of Theorem 7.1 (Rewrite Soundness)

Proof. Let Q, Q' be queries over schema $\mathcal{S}$ whose encoding lies within $\mathcal{F}_{\text{exact}}$, and let $\Sigma = (K, \mathcal{D}, v)$. Assume SynthesizeWitness returns Unsat.

The solver allocates K symbolic rows per table, each carrying a *present* flag (Lemma A.1). At cardinality K, the formula $\phi_{\mathcal{D}}^{(K)} \wedge \Delta^{(K)}$ is unsatisfiable, where $\phi_{\mathcal{D}}^{(K)}$ constrains the *present* flags and value domains, and $\Delta^{(K)}$ is the difference predicate.

By Lemma A.1, the *present* flags allow a single call at cardinality K to represent *every* instance with at most K rows per table, including mixed-cardinality instances and 0-row tables. By Lemma A.7, the unsatisfiability of $\phi_{\mathcal{D}}^{(K)} \wedge \Delta^{(K)}$ implies that no instance $D \in \mathsf{Inst}(\Sigma, \mathcal{S})$ distinguishes Q from Q'.

Therefore $\{\!\{Q\}\!\}_D = \{\!\{Q'\}\!\}_D$ for all $D \in \mathsf{Inst}(\Sigma, \mathcal{S})$, i.e., $Q \equiv_\Sigma Q'$.

(The ascending dense schedule $k = 1, \ldots, K$ provides incremental results and early termination; the coverage argument requires only the k=K step.) □

A.3 Proof of Proposition 5.1 (Safety of Fresh-Variable Encoding)

Proof. Fresh variables *enlarge* the solution space: every satisfying assignment of Φ^* extends to a satisfying assignment of Φ (by choosing the fresh variables to match the exact semantics), so $\textsc{Unsat}(\Phi) \Rightarrow \textsc{Unsat}(\Phi^*)$. Conversely, Φ may admit assignments where the fresh variables take values inconsistent with the true function semantics, yielding a spurious witness. □

A.4 Proof of Theorem 7.2 (Conservative Approximation Safety)

Proof. Let $e_1, \ldots, e_m$ be the subexpressions in Q or Q' that are outside $\mathcal{F}_{\text{exact}}$ and are encoded via fresh unconstrained Z3 variables $x_1, \ldots, x_m$ of matching sorts.

Let Φ denote the formula produced by CertOpt (with fresh variables) and Φ^* denote the hypothetical exact formula. We show that $\textsc{Unsat}(\Phi) \Rightarrow \textsc{Unsat}(\Phi^*)$.

Step 1: Solution-space inclusion. Under exact encoding, each e_i would be constrained by the functional relationship $x_i = f_i(\bar{y}_i)$ where f_i is the SQL semantic function and $\bar{y}_i$ are the input variables. Under fresh-variable encoding, x_i is unconstrained in its sort domain. Therefore, every satisfying assignment of Φ^* can be extended to a satisfying assignment of Φ by choosing $x_i = f_i(\bar{y}_i)$, establishing:

$$Sol(\Phi^*) \subseteq Sol(\Phi)$$

where $Sol(\cdot)$ denotes the set of satisfying assignments projected onto the shared variables.

Step 2: UNSAT preservation. If $Sol(\Phi) = \emptyset$ (UNSAT), then $Sol(\Phi^*) \subseteq Sol(\Phi) = \emptyset$, so Φ^* is also UNSAT. The hypothetical exact formula Φ^* encodes queries entirely within $\mathcal{F}_{\text{exact}}$ (the unmodeled subexpressions are, by definition, given their true semantic functions in Φ^*). Therefore Lemma A.7 applies to Φ^*: its unsatisfiability implies that no distinguishing instance exists within the bounded scope Σ.

Step 3: SAT direction. If $Sol(\Phi) \neq \emptyset$ (SAT), a satisfying assignment may assign fresh variables to values inconsistent with f_i. Such a witness is *spurious*. The witness validation pipeline (§7.2) catches these by executing both queries on the materialized database.

Key property: Each occurrence of an unmodeled subexpression receives an *independent* fresh variable, keyed by AST node identity. Even syntactically identical subexpressions in Q and Q' receive distinct variables, ensuring that the relaxation cannot artificially correlate the two queries' evaluations.

Exclusion: This argument applies only to *value relaxations* (replacing a value expression with an unconstrained variable of matching sort). Predicate *strengthenings*—such as the equality fallback for non-literal `LIKE`—do not satisfy the solution-space inclusion in Step 1 and are therefore excluded from this theorem. Such cases are reported as Unknown. □

A.5 Proof of Theorem 7.3 (Bounded Optimality)

Proof. Let C be the set of all candidates generated by the rewrite generator, and let

$$C_{\text{eq}} = \left\{ Q' \in C \mid \textsc{SynthesizeWitness}(Q, Q', \mathcal{S}, \Sigma) = \textsc{Unsat} \right\}$$

be the subset of candidates that were individually verified equivalent (not merely explored or pruned).

Semantic preservation. By Theorem 7.1 (for exact-fragment queries) or Theorem 7.2 (for queries with fresh-variable approximations), every $Q' \in C_{\text{eq}}$ satisfies $Q' \equiv_\Sigma Q$.

Cost optimality. Algorithm 1 selects $Q^* = \arg\min_{Q' \in C_{\text{eq}}} \text{cost}(Q')$ and emits Q^* only if $\text{cost}(Q^*) < \text{cost}(Q)$.

Therefore:

(1) $Q^* \equiv_\Sigma Q$ (semantic preservation under Σ).
(2) $\text{cost}(Q^*) \leq \text{cost}(Q')$ for all $Q' \in C_{\text{eq}}$ (cost-optimal within the set of candidates that were *individually certified*).

The guarantee is bounded both semantically (by Σ) and in search coverage (by the generator C). Candidates eliminated by family pruning are excluded from C_{eq} and do not affect the guarantee: pruning may cause missed optimizations but cannot introduce unsound acceptances, since only individually verified candidates enter C_{eq}. □

A.6 Proof of Theorem 6.1 (INNER JOIN Predicate Relocation)

We split the proof into two parts: a local algebraic lemma and a contextual substitutivity theorem, then compose them.

Lemma A.8 (Local Predicate Relocation for INNER JOIN). *Under INNER JOIN semantics and SQL's three-valued logic, for any predicate φ and join predicate θ:*

$$\sigma_\varphi(L \bowtie_\theta R) = L \bowtie_{\theta \wedge \varphi} R$$

where equality is bag equality over the full schema of L and R.

Proof. A tuple (l, r) appears in $\sigma_\varphi(L \bowtie_\theta R)$ iff $tb\_true(\theta(l, r))$ and $tb\_true(\varphi(l, r))$ both hold. It appears in $L \bowtie_{\theta \wedge \varphi} R$ iff $tb\_true((\theta \wedge \varphi)(l, r))$ holds.

Under 3VL AND semantics (Lemma A.3):

$$tb\_true(\theta \wedge \varphi) = tb\_true(\theta) \wedge tb\_true(\varphi)$$

because $tb\_true(\langle u, v \rangle) = \neg u \wedge v$, so definite truth distributes over conjunction (Kleene AND).

Therefore the two expressions admit exactly the same tuples with the same multiplicities. □

Lemma A.9 (Bag-Contextual Substitutivity). *Let $C[\cdot]$ be a* typed one-hole context*: a query with a single hole of relational type, where the hole produces tuples over interface schema I. We call $C[\cdot]$ an* admissible context *if it satisfies two conditions: (i) every reference to an alias defined inside the hole B appears only via columns in I; and (ii) predicates in C may reference columns of I together with columns of outer-context aliases, but may* not *reference internal aliases of B that are not exposed through I. (Condition (ii) is verified by the predicate-closure check in §6.1; note that a join predicate $[\cdot] \bowtie_\theta T$ referencing I-columns and T-columns is admissible because it observes B only through I.)*

If B_1 and B_2 produce the same bag over schema I for all $D \in \mathsf{Inst}(\Sigma, \mathcal{S})$—i.e., $\pi_I(B_1)_D = \pi_I(B_2)_D$ for all $D \in \Sigma$—then $C[B_1] \equiv_\Sigma C[B_2]$.

Proof. By structural induction on $C[\cdot]$, we show $\{\!\{C[B_1]\}\!\}_D = \{\!\{C[B_2]\}\!\}_D$ for all D.

Admissibility. The predicate-closure check ensures that every predicate in C referencing an alias from B is fully contained in I. Therefore, the only way C observes B is through the I-projected tuples.

Base case: $C = \pi_J[\cdot]$ (projection). Since $J \subseteq I$ (the output columns are a subset of the interface columns), $\pi_J(B_1)_D = \pi_J(B_2)_D$ follows directly from $\pi_I(B_1)_D = \pi_I(B_2)_D$.

Inductive cases. (i) *Selection σ_ψ: ψ* references only I-columns (by admissibility). Since $\pi_I(B_1)_D = \pi_I(B_2)_D$ as bags, applying the same deterministic filter ψ to each yields the same output bag.

(ii) *Join $[\cdot] \bowtie_\theta T$:* Each tuple in $\pi_I(B)_D$ is paired with each row of T in D. Since the bags of I-tuples are identical and θ depends only on I-columns and T-columns (by admissibility), the join produces the same output bag.

(iii) *Aggregation $\gamma_{G,f}$:* The GROUP BY keys G and aggregate expressions f reference only I-columns (by admissibility). Since the input bags are identical, the groups are identical, and the aggregate values within each group are identical.

(iv) *ORDER BY + LIMIT:* The ORDER BY keys reference only I-columns. Since the input bags are identical, the set of possible orderings (accounting for SQL's underspecified tie behavior) is identical, and hence the set of possible top-ℓ result bags is identical.

(v) *DISTINCT:* Deduplication of identical bags yields identical sets.

(vi) *Composition:* Multiple operators compose by induction: the output of each operator is used as input to the next, and bag equality is preserved at each step. □

Proof of Theorem 6.1. Combine Lemma A.8 and Lemma A.9.

Step 1. By Lemma A.8, $\sigma_\varphi(L \bowtie_\theta R) = L \bowtie_{\theta \wedge \varphi} R$ as bags over the full schema. Projecting both sides onto I (which includes all boundary-alias columns) preserves bag equality: $\pi_I(B_1)_D = \pi_I(B_2)_D$ for all D.

Step 2. The decomposition algorithm (§6.1) constructs local queries B_1, B_2 projected onto I and verifies $\pi_I(B_1) \equiv_\Sigma \pi_I(B_2)$ via the bounded equivalence checker. The predicate-closure check ensures the outer context $C[\cdot]$ is admissible.

Step 3. By Lemma A.9, $C[B_1] \equiv_\Sigma C[B_2]$, i.e., $Q[B_1] \equiv_\Sigma Q[B_2]$.

Multiple regions. For m independent predicate relocations on m distinct joins with *pairwise disjoint* local alias sets, compose by repeated application: $Q_0 \to Q_1 \to \cdots \to Q_m$, where each step $Q_{i-1} \to Q_i$ substitutes one verified-equivalent local block. Disjointness ensures that each later substitution does not alter a previously verified local block, so each step is an independent application of the single-region argument above. Each substitution preserves Σ-equivalence, so $Q_0 \equiv_\Sigma Q_m$ by transitivity.

Local SAT is inconclusive. If a local region returns Sat, the distinguishing witness applies to the isolated local query $\pi_I(B_1) \neq \pi_I(B_2)$ on some D^*. However, the outer context $C[\cdot]$ may filter, aggregate, or join away the distinguishing tuples, so $C[B_1]_{D^*}$ and $C[B_2]_{D^*}$ may still agree. Therefore local Sat does *not* imply full-query non-equivalence; it is conservatively treated as inconclusive.

Bound requirement. The compositional acceptance guarantee (Corollary 6.1) requires every local region to be verified at bound Σ. In the implementation, large local regions ($\geq$5 tables) are checked at a reduced bound k=1 as a practical concession to encoding tractability (§6.1). In such cases the compositional proof establishes equivalence only at the minimum local bound, not the full global Σ. This weakening does not affect soundness—the system never accepts a non-equivalent rewrite—but the certified bound may be smaller than the nominal Σ. □

A.7 Proof of Proposition 6.1 (Combinatorial Reduction)

Proof. Let Q be a query over n base tables, verified at bound k.

Monolithic cost. The monolithic encoding constructs k symbolic rows per table and enumerates all row combinations (combos). Each combo selects one of k rows from each of the n tables, yielding k^n total combos. The Z3 formula contains $O(k^n)$ survival predicates. For aggregated queries, the same-group matrix (Definition 5.4) has $O(k^{2n})$ entries, and the difference predicate (Definition 5.6) compares $O(k^n)$ result rows pairwise.

Compositional cost. Under per-join decomposition (§6.1), each of the m local regions contains n_i tables and is verified independently (Corollary 6.1), enumerating k^{n_i} combos. The total is $\sum_{i=1}^{m} k^{n_i}$.

Reduction bound. Since each $n_i \leq n_{\max}$ (where $n_{\max}$ is the largest local region, bounded by the $|local| \leq 6$ threshold):

$$\sum_{i=1}^{m} k^{n_i} \;\leq\; m \cdot k^{n_{\max}}$$

Therefore the reduction factor satisfies:

$$\frac{k^n}{\sum_{i=1}^{m} k^{n_i}} \;\geq\; \frac{k^n}{m \cdot k^{n_{\max}}} \;=\; \frac{k^{n-n_{\max}}}{m}$$

Since $n_{\max} \ll n$ in practice (typically $n_{\max} \leq 3$ for predicate-relocation rewrites while $n \geq 7$), this factor grows *exponentially* in $n - n_{\max}$.

Super-linear amplification. The encoding cost is not merely linear in the combo count. For aggregated queries, the same-group matrix has $O(c^2)$ entries where c is the combo count, making the encoding time super-linear. The exponential combo reduction thus translates to an even larger wall-clock speedup, as confirmed empirically in §6.4. □

B SQL CONSTRUCT COVERAGE

Table 11 summarizes the SQL constructs supported by CertOpt, their representation in the typed IR, and the encoding quality in witness synthesis.

C THREE-VALUED LOGIC TRUTH TABLES

The connectives $\wedge$, $\vee$, and $\neg$ under Kleene's three-valued logic are defined as follows (U abbreviates UNKNOWN):

$\wedge$	T	F	U
T	T	F	U
F	F	F	F
U	U	F	U

$\vee$	T	F	U
T	T	T	T
F	T	F	U
U	T	U	U

$\neg$	
T	F
F	T
U	U

Table 11: SQL construct coverage in CertOpt. "Exact" means the Z3 encoding faithfully models SQL bag semantics for the construct; for tie-sensitive constructs (`ORDER BY ... LIMIT`, ranking windows), "Exact" denotes sound coverage of all legal tie resolutions (§H). "Approx." means a conservative approximation is used (safe direction: false SAT possible, false UNSAT impossible).

Construct	Parser	IR Node	Renderer	Synthesis	Encoding
`SELECT / FROM / WHERE / JOIN`	✓	`QueryIR`	✓	Exact	Bounded bag semantics
`GROUP BY / HAVING`	✓	`QueryIR`	✓	Exact	Grouping matrix
`ORDER BY / LIMIT`	✓	`QueryIR`	✓	Exact	Rank-based top-k
`DISTINCT`	✓	`QueryIR`	✓	Exact	Bag → set dedup
`INNER JOIN`	✓	`JoinClause`	✓	Exact	Filter-based
`LEFT / RIGHT / CROSS JOIN`	✓	`JoinClause`	✓	Exact	Outer-join NULL padding
Derived tables	✓	`DerivedTable`	✓	Exact	Inline expansion
`EXISTS / NOT EXISTS`	✓	`ExistsSubquery`	✓	Exact	OR of inner surviving rows
`IN` (subquery)	✓	`InSubquery`	✓	Exact	3VL match with NULL handling
Scalar subquery	✓	`ScalarSubquery`	✓	Exact	ITE: first surviving row
Correlated subqueries	✓	(outer ref injection)	✓	Exact	Outer binding injection
`UNION ALL`	✓	`set_op`	✓	Exact	Row concatenation
`UNION`	✓	`set_op`	✓	Exact	Concat + first-occurrence dedup
`INTERSECT`	✓	`set_op`	✓	Exact	Rank-based min multiplicity
`EXCEPT`	✓	`set_op`	✓	Exact	Rank-based $\max(0, \text{diff})$
`IN` (list)	✓	`InList`	✓	Exact	3VL membership OR-chain
`BETWEEN`	✓	`Between`	✓	Exact	Range comparison
`CASE / WHEN`	✓	`CaseExpr`	✓	Exact	Conditional ITE chain
`IS [NOT] NULL`	✓	`UnaryOp`	✓	Exact	Nullability check
`LIKE`	✓	`BinOp(LIKE)`	✓	Approx.	Equality approximation
String/date functions	✓	`FuncCall`	✓	Approx.	Fresh unconstrained variable
Window functions (exact encoding; tie-resolution semantics argued in Appendix H)					
`ROW_NUMBER()`	✓	`WindowFunc`	✓	Exact	Partition rank + existential tiebreakers
`RANK()`	✓	`WindowFunc`	✓	Exact	1+ count of strictly better peers
`DENSE_RANK()`	✓	`WindowFunc`	✓	Exact	1+ distinct better key-tuples
`COUNT/SUM/AVG/MIN/MAX OVER`	✓	`WindowFunc`	✓	Exact	Partition-based aggregate reuse
`LAG / LEAD / FIRST_VALUE`	✓	`WindowFunc`	✓	Exact	Position-based value access
`ROWS BETWEEN` frames	✓	`WindowFunc`	✓	Exact	Frame membership via position bounds
Recursive CTEs	—	—	—	—	Not supported
Lateral joins	—	—	—	—	Not supported

D ORDER BY WITH LIMIT ENCODING

For each surviving result row r, define its rank: $rank(r) = 1 + |\{s \neq r : survives(s) \wedge s \prec_{ord} r\}|$. Row r survives the LIMIT phase iff $rank(r) \leq \ell$. The comparison $\prec_{ord}$ respects SQL null ordering: for `ASC`, non-null values precede `NULL` (`NULLS LAST`); for `DESC`, `NULL` precedes non-null (`NULLS FIRST`). Ties are broken by fresh unconstrained integer variables tb_i, letting Z3 existentially quantify over all possible tie resolutions.

E EXTENDED TECHNICAL DETAILS

E.1 Concrete Z3 Encoding Example

We illustrate the full SMT encoding for a simple Calcite pair (pair 134) at k=1. Schema: `EMP(empno PK, ename, deptno NOT NULL)`, `DEPT(deptno PK, name)`.

```
-- Q1 (explicit JOIN):
SELECT dept.name, emp.ename
FROM emp INNER JOIN dept
  ON emp.deptno = dept.deptno
WHERE dept.name = 'Propane'
```

```
-- Q2 (comma join):
SELECT dept.name, emp.ename
FROM emp, dept
WHERE emp.deptno = dept.deptno
  AND dept.name = 'Propane'
```

At k=1, CertOpt creates one symbolic row per table. Each column becomes a pair of Z3 variables: a value (_v) and a null flag (_n). The solver produces 12 assertions in three groups. All formulas below are in SMT-LIB2 syntax (S-expression prefix notation), the standard input format for Z3:

Group 1: Domain constraints (assertions 0–10). Non-nullable columns are forced non-null; integer values are bounded to $[-10, 10]$; string values are mapped to a finite symbol table (integers 0, 1, 2 representing distinct string constants).

```
; dept.deptno: NOT NULL, integer bounds
(not dept_deptno_0_n)
(=> (not dept_deptno_0_n)
    (and (>= dept_deptno_0_v -10)
```

```
        (<= dept_deptno_0_v 10)))
(=> (not dept_deptno_0_n)
    (is_int dept_deptno_0_v))

; dept.name: nullable, string → symbol index
(=> (not dept_name_0_n)
    (and (>= dept_name_0_v 0)
         (< dept_name_0_v 3)))

; emp.empno, emp.deptno: NOT NULL, bounded
(not emp_empno_0_n)
(not emp_deptno_0_n)
; ... (analogous bounds)

; emp.ename: nullable, string → symbol index
(=> (not emp_ename_0_n)
    (and (>= emp_ename_0_v 0)
         (< emp_ename_0_v 3)))
```

Group 2: Query encoding (assertion 11, part 1). The join condition and WHERE predicate are encoded identically for both queries—`a!1` captures the surviving row:

```
; a!1 = row survives (both queries agree):
(and true                            ; FROM
     true                            ; no extra ON
     (not (or emp_deptno_0_n         ; join: not null
              dept_deptno_0_n))
     (= emp_deptno_0_v               ; join: equal
        dept_deptno_0_v)
     (not (or dept_name_0_n false)) ; WHERE: not null
     (= 1 dept_name_0_v))          ; WHERE: = 'Propane'
                                    ;  (symbol 1)
```

Group 3: Difference predicate (assertion 11, part 2). The bag-disagreement formula checks whether any result tuple has different multiplicity between Q_1 and Q_2. Both queries produce identical result expressions, so the difference is trivially unsatisfiable:

```
; a!2 = result-row match (both-null-or-equal
;        per projected column):
(and a!1
     (or (and dept_name_0_n dept_name_0_n)
         (and (not dept_name_0_n)
              (not dept_name_0_n)
              (= dept_name_0_v dept_name_0_v)))
     (or (and emp_ename_0_n emp_ename_0_n)
         (and (not emp_ename_0_n)
              (not emp_ename_0_n)
              (= emp_ename_0_v emp_ename_0_v))))

; Difference: multiplicity(Q1) != multiplicity(Q2)
(and a!1
     (distinct (+ (ite a!2 1 0))   ; count in Q1
               (+ (ite a!2 1 0)))) ; count in Q2
```

Z3 returns Unsat in 0.1 ms: the two queries produce identical symbolic expressions, so no database instance within Σ can distinguish them. This confirms that explicit `JOIN ON` and comma-join with equivalent `WHERE` predicates are semantically identical under bag semantics—a Calcite optimizer rule that CertOpt certifies formally.

E.2 Bounded Equivalence: Decidability Caveat

If a distinguishing witness exists within scope Σ, the two queries are provably inequivalent (on at least that instance). If no witness exists within Σ, bounded equivalence holds but *full* equivalence is not guaranteed—the queries may still disagree on larger instances outside the scope. The key insight that makes bounded verification practical is that it is *decidable*: for a fixed scope Σ, the search for a distinguishing witness can be reduced to a satisfiability query in a decidable theory and dispatched to an SMT solver such as Z3 [13].

E.3 Monolithic Scaling Limits

Without compositional decomposition, monolithic verification cannot decide *any* of the 30 JOB-Complex queries: every query exceeds the monolithic combination limit (the smallest has 7 tables, yielding $2^7 = 128$ row combinations at k=2 and $3^7 = 2{,}187$ at k=3). The concrete example JOB-C01 (n=9 tables, m=5 moved predicates) demonstrates the reduction: monolithic $= 2^9 = 512$ combos; compositional $= 2{+}4{+}4{+}4{+}4 = 18$ (five regions with $n_i \in \{1, 2, 2, 2, 2\}$), a 28× reduction. At k=3 the gap widens: $3^9 = 19{,}683$ vs. $3{+}9{+}9{+}9{+}9 = 39$, a 505× reduction. The median reduction across all 27 verified queries is 85×.

E.4 Witness Validation Pipeline

The three stages of witness validation are:

Stage 1: Witness materialization. Extract concrete table contents from the Z3 model, decode symbolic integers back to their original-domain values (strings via the symbol table, dates via epoch offsets), and materialize the witness database in SQLite (or DuckDB for queries requiring window functions or advanced types).

Stage 2: Differential execution. Execute both the original query Q and the rewrite Q' on the materialized witness database. Compare the result sets as *multisets*: two bags agree if and only if every tuple appears with identical multiplicity in both.

Stage 3: Verdict adjudication. If the results disagree, the witness is *confirmed* and the rewrite is definitively rejected. If the results agree, the witness is *spurious*: the verdict is downgraded from Sat to Unknown, and the rewrite is treated as unverified rather than rejected.

CEGAR refinement. When a spurious witness is detected, CertOpt adds a blocking constraint to the Z3 context that excludes the spurious model and re-invokes the solver. This counterexample-guided abstraction refinement (CEGAR) loop runs for up to 3 iterations. If all iterations produce spurious witnesses, the final verdict is Unknown; if any iteration produces a confirmed witness, the rewrite is rejected. In practice, CEGAR refinement resolves the majority of spurious witnesses within 1–2 iterations.

Illustrated: Confirmed witness (Calcite, pair 201). CertOpt rejects the following Calcite optimizer rewrite that expands statistical aggregates into arithmetic:

```
-- Q1 (original):
SELECT NAME, STDDEV_POP(DEPTNO), AVG(DEPTNO)
FROM DEPT GROUP BY NAME

-- Q2 (Calcite rewrite, expanded):
SELECT NAME,
```

```
  CAST(POWER((...SUM/COUNT expansion...), 0.5)
    AS INTEGER),
  CAST(SUM(DEPTNO)/COUNT(*) AS INTEGER)
FROM DEPT GROUP BY NAME
```

Stage 1 materializes the Z3 model into DuckDB (or SQLite) with two DEPT rows: (deptno=1, name='A') and (deptno=2, name='A'). **Stage 2** executes both queries: Q_1 returns STDDEV_POP = 0.5, AVG = 1.5; Q_2 returns CASTed integers 0 and 1 due to CAST(... AS INTEGER) truncation. **Stage 3**: results disagree → witness *confirmed*, verdict NEQ. The root cause is that Calcite's expansion uses integer CAST on intermediate results, losing fractional precision.

Illustrated: Spurious witness (Calcite, pair 33). CERTOPT encounters a SAT result for this COUNT(DISTINCT) decomposition:

```
-- Q1:
SELECT DEPTNO, COUNT(DISTINCT ENAME),
       COUNT(DISTINCT JOB, ENAME), SUM(SAL)
FROM EMP GROUP BY DEPTNO

-- Q2 (Calcite rewrite):
SELECT t1.DEPTNO, t3.EXPR$1, t5.EXPR$2, t1.EXPR$4
FROM (SELECT DEPTNO, SUM(SAL) AS EXPR$4
      FROM EMP GROUP BY DEPTNO) AS t1
  JOIN (...COUNT subqueries...) AS t3 ...
  JOIN (...COUNT subqueries...) AS t5 ...
```

The solver produces a witness, but **Stage 2** executes both queries on it and finds identical results—the witness is *spurious*. The solver over-approximated because COUNT(DISTINCT col1, col2) is outside the exactly modeled fragment (multi-column DISTINCT aggregation). **Stage 3** downgrades the verdict to UNKNOWN. VeriEQL also cannot handle this pair (returns ERR).

E.5 VeriEQL False Equivalence Proof Root Causes

The 241 confirmed VeriEQL false equivalence proofs on LeetCode trace to five root causes in VeriEQL's source code:
(i) Integer division as real division (153 pairs). VeriEQL dispatches SQL's / through Python's operator.true_div (real division), missing integer truncation semantics. Queries involving DURATION/60 or AVG(RATING/POSITION) produce different results under integer vs. real division (problems 1211, 1435, 1468, 585, 1731).
(ii) Conflicting primary-key constraints (30 pairs). Problem 1789 specifies two PK constraints on the same table: (employee_id, department_id) (composite) and (employee_id) (single). VeriEQL's constraint encoder (environment.py:627–644) enforces both independently—the single-column PK subsumes the composite, making HAVING COUNT(*)=1 vacuously true and collapsing semantic distinctions between the two queries.
(iii) Column pivot NULL/duplicate semantics (34 pairs). SUM(CASE WHEN store='X' THEN price END) vs. multi-way LEFT JOIN pivots produce different results when rows have NULL values or duplicate store entries. VeriEQL proves equivalence but the queries differ on these edge cases (problems 1795, 1777).
(iv) NOT IN *with NULL propagation (15 pairs).* NOT IN (subquery) returns the empty set when the subquery contains NULL—a well-known three-valued-logic pitfall. VeriEQL raises NotSupportedError for some cases but handles others without correct NULL propagation (problems 1083, 1264).
(v) IF() function and structural encoding gaps (9 pairs). MySQL's IF() in JOIN conditions changes join semantics (returns NULL when false, preventing the match). Additionally, complex nested CASE WHEN with string comparisons exposes encoding gaps (problems 1378, 1440, 584).

F PREPROCESSING ALGORITHMS

Before witness synthesis, CERTOPT applies four preprocessing passes that reduce the number of tables and simplify predicates, thereby shrinking the combinatorial search space. Each pass preserves query semantics under the conditions checked.

F.1 Pass 1: Predicate-to-ON Promotion

For each top-level conjunct φ in the WHERE clause that is an equi-join predicate $t_1.c_1 = t_2.c_2$, if t_2 is on the right side of a CROSS or INNER JOIN, move φ to the join's ON clause and convert CROSS JOIN to INNER JOIN.

This pass converts implicit-join SQL (comma-separated FROM with WHERE predicates) to explicit-join form. The resulting query has the same semantics under bag semantics for INNER and CROSS joins.

Effect on verification: CROSS JOINs produce all k^n combinations. Converting to INNER JOINs allows the combo enumerator to generate only combinations where the ON predicate holds, reducing the effective search space.

F.2 Pass 2: Redundant Table Elimination

Iteratively (until fixed point), for each INNER-JOINed table T:

(1) Check that T is not referenced in SELECT, WHERE, GROUP BY, HAVING, or ORDER BY.
(2) Check that T is not referenced in any other join's ON clause.
(3) Check that the join predicate matches an FK→PK relationship with NOT NULL on the FK column.
(4) If all conditions hold, remove the join.

The FK→PK NOT NULL guarantee ensures that each left-side row matches *exactly one* right-side row under INNER JOIN. Since the right-side table contributes no columns to the output and does not appear in any predicate, its removal is a semantics-preserving transformation. The iterative pass handles cascaded elimination (e.g., removing c first makes b eliminable).

F.3 Pass 3: Existence-Only Table Rewriting

Identifies INNER-JOINed tables whose columns appear *only* in WHERE or ON predicates (not in SELECT, GROUP BY, HAVING, or ORDER BY). When the query uses DISTINCT (making multiplicities irrelevant), such a table can be rewritten as an EXISTS subquery:

```
-- Before:
SELECT DISTINCT a.val FROM a JOIN b ON a.b_id = b.id
WHERE b.status = 'active'
```

```
-- After:
SELECT DISTINCT a.val FROM a
WHERE EXISTS (SELECT 1 FROM b WHERE a.b_id = b.id
              AND b.status = 'active')
```

This reduces the combo count because the EXISTS subquery is evaluated independently for each outer row rather than contributing to the cartesian product.

F.4 Pass 4: Implied-Predicate Join Elimination

When a table T is joined via FK→PK and its columns appear *only* in WHERE predicates, those predicates can be *transplanted* to reference the FK column on the other side of the join:

```
-- Before:
SELECT a.val FROM a JOIN b ON a.b_id = b.id WHERE b.id > 5

-- After (transplant b.id -> a.b_id):
SELECT a.val FROM a WHERE a.b_id > 5
```

The join and all references to T are eliminated. This is sound because the FK→PK equality guarantee ($a.b\_id = b.id$) means that any predicate on $b.id$ can equivalently be expressed on $a.b\_id$.

G WORKED EXAMPLES

G.1 Example 1: Predicate Pushdown into JOIN (Calcite)

Source: Calcite optimizer test suite (VeriEQL benchmark [19], pair 71).
Query 1:

```
SELECT 1
FROM EMP
  INNER JOIN EMP AS EMP0 ON EMP.DEPTNO = EMP0.DEPTNO
  INNER JOIN EMP AS EMP1 ON EMP0.DEPTNO = EMP1.DEPTNO
WHERE EMP.DEPTNO > 7
```

Query 2:

```
SELECT 1
FROM (SELECT * FROM EMP WHERE DEPTNO > 7) AS t1
  INNER JOIN (SELECT * FROM EMP WHERE DEPTNO > 7) AS t2
    ON t1.DEPTNO = t2.DEPTNO
  INNER JOIN (SELECT * FROM EMP WHERE DEPTNO > 7) AS t3
    ON t2.DEPTNO = t3.DEPTNO
```

Result: Equ in 698 ms at k=3.
Analysis. Calcite pushes the WHERE predicate `DEPTNO > 7` into each leg of the self-join. Because all three joins are INNER and the predicate references only DEPTNO (the join key), the filter can be safely distributed. CertOpt verifies the bag-semantic equivalence at k=3.

G.2 Example 2: JOIN-to-EXISTS Decorrelation (Calcite)

Source: Calcite optimizer test suite (VeriEQL benchmark [19], pair 16).
Query 1:

```
SELECT t.DEPTNO AS DEPARTMENT_ID, t.SAL AS SALARY
FROM (SELECT SAL, DEPTNO FROM EMP) AS t
  INNER JOIN (SELECT DEPTNO FROM DEPT) AS t0
    ON t.DEPTNO = t0.DEPTNO
```

Query 2:

```
SELECT DEPTNO AS DEPARTMENT_ID, SAL AS SALARY
FROM (SELECT SAL, DEPTNO FROM EMP
      WHERE EXISTS (SELECT 1
        FROM (SELECT DEPTNO FROM DEPT) AS t3
        WHERE DEPTNO = t3.DEPTNO)) AS t2
```

Result: Equ in 38 ms at k=3.
Analysis. Calcite rewrites an `INNER JOIN` with a derived table into an `EXISTS` subquery. The transformation preserves bag semantics because `DEPT.DEPTNO` is a primary key, guaranteeing at most one match per EMP row. CertOpt proves equivalence at k=3 in 38 ms.

G.3 Example 3: FULL JOIN to INNER JOIN Simplification (Calcite)

Source: Calcite optimizer test suite (VeriEQL benchmark [19], pair 209).
Query 1:

```
SELECT 1
FROM DEPT FULL JOIN EMP ON DEPT.DEPTNO = EMP.DEPTNO
WHERE DEPT.NAME = 'Charlie' AND EMP.SAL > 100
```

Query 2:

```
SELECT 1
FROM (SELECT * FROM DEPT WHERE NAME = 'Charlie') AS t1
  INNER JOIN (SELECT * FROM EMP WHERE SAL > 100) AS t2
    ON t1.DEPTNO = t2.DEPTNO
```

Result: Equ in 183 ms at k=3.
Analysis. When WHERE predicates filter on non-NULL columns from both sides of a `FULL OUTER JOIN`, the unmatched (NULL-padded) rows are always eliminated, making the `FULL JOIN` equivalent to an `INNER JOIN`. CertOpt verifies this optimization under bag semantics.

G.4 Example 4: IS NOT DISTINCT FROM (Calcite, VeriEQL=ERR)

Source: Calcite optimizer test suite (VeriEQL benchmark [19], pair 389). VeriEQL returns ERR on this pair.
Query 1:

```
SELECT *
FROM EMP
  INNER JOIN EMP AS EMP0 ON EMP.DEPTNO = EMP0.DEPTNO
WHERE EMP.ENAME IS NULL AND EMP0.ENAME IS NULL
   OR EMP.ENAME = EMP0.ENAME IS TRUE
```

Query 2:

```
SELECT *
FROM EMP AS EMP1
  INNER JOIN EMP AS EMP2 ON EMP1.DEPTNO = EMP2.DEPTNO
    AND EMP1.ENAME IS NOT DISTINCT FROM EMP2.ENAME
```

Result: Equ in 832 ms at k=3.
Analysis. The original uses a verbose NULL-aware equality pattern; the rewrite uses SQL's `IS NOT DISTINCT FROM`. VeriEQL cannot handle `IS NOT DISTINCT FROM` (returns ERR). CertOpt models the three-valued semantics correctly and proves equivalence, demonstrating broader SQL dialect coverage.

G.5 Example 5: Compositional Verification (JOB-C01)

Source: JOB-Complex benchmark [40].

JOB-C01 joins 9 tables. The original query applies all filter predicates in the WHERE clause:

```
SELECT MIN(mc.note), MIN(t.title), MIN(t.production_year)
FROM company_type ct, title t, movie_companies mc,
    cast_info ci, company_name cn, name n,
    movie_info mi, movie_keyword mk, keyword k
WHERE ct.kind = 'production companies'
  AND cn.country_code = '[us]'
  AND k.keyword = 'sequel'
  AND t.id = mc.movie_id AND mc.company_id = cn.id
  AND mc.company_type_id = ct.id AND ci.movie_id = t.id
  AND ci.person_id = n.id AND mi.movie_id = t.id
  AND mk.movie_id = t.id AND mk.keyword_id = k.id
```

Rule R1 (predicate pushdown) moves each filter predicate into the ON clause of the join that introduces the filtered table. For instance, `ct.kind = 'production companies'` moves from WHERE into `mc JOIN ct ON ... AND ct.kind = 'production companies'`.

Monolithic verification at k=2 encodes all 9 tables, producing $2^9 = 512$ row combinations in a single Z3 formula. This exceeds the combo limit and returns UNKNOWN.

Per-join decomposition isolates each moved predicate into a small local region:

(1) **Detect moved predicates:** 5 predicates migrated from WHERE to 5 different ON clauses.

(2) **Build local region** (e.g., for `ct.kind`): the changed join is `mc JOIN ct`; local aliases = $\{mc, ct\}$ (2 tables, not 9).

(3) **Identify boundary:** mc is referenced in other joins and SELECT, so it is a boundary alias. Export all columns of mc as the interface I.

(4) **Build local query pair:**

```
-- Local Q (predicate in WHERE):
SELECT mc.* FROM movie_companies mc
  JOIN company_type ct ON mc.company_type_id = ct.id
WHERE ct.kind = 'production companies'

-- Local Q' (predicate in ON):
SELECT mc.* FROM movie_companies mc
  JOIN company_type ct ON mc.company_type_id = ct.id
    AND ct.kind = 'production companies'
```

(5) **Verify locally:** 2 tables at k=2 $\rightarrow$ $2^2 = 4$ row combinations $\rightarrow$ Z3 returns UNSAT in milliseconds.

(6) **Compose:** By Theorem 6.1, since the local region produces the same bag of `mc.*` tuples with the predicate in either position, swapping the predicate in the full 9-table query preserves the result.

All 5 local regions (each 2–3 tables) return UNSAT. Total row combinations: $5 \times 4 = 20$ vs. 512 monolithic—a **25**× reduction. At k=3 the gap widens to $5 \times 9 = 45$ vs. $3^9 = 19{,}683$, a **437**× reduction.

G.6 Example 6: LLM Rewrite Verification

Source: JOB-Complex benchmark [40], LLM-generated rewrite.

Original:

```
SELECT MIN(cn.name)
FROM company_name cn, title t, movie_companies mc
WHERE cn.id = mc.company_id
  AND t.id = mc.movie_id
  AND t.production_year > 2000
```

LLM-proposed rewrite:

```
SELECT MIN(cn.name)
FROM company_name cn
JOIN movie_companies mc ON cn.id = mc.company_id
JOIN title t ON t.id = mc.movie_id
WHERE t.production_year > 2000
```

Pipeline: (1) Parse $\rightarrow$ QueryIR. (2) Structural verify: schema valid, types sound. (3) Preprocess: promote predicates (already explicit). (4) Witness synthesis: UNSAT (equivalent under Σ). (5) Cost: 340.2 (original) vs. 290.5 (rewrite) $\rightarrow$ 1.17× speedup. (6) Certificate generated $\rightarrow$ accept.

G.7 Example 7: VeriEQL Spurious Refutation (NATURAL JOIN)

Source: VeriEQL Literature benchmark [19].

During our cross-comparison on the Literature benchmark, we found one pair where VeriEQL reports NEQ but CertOpt proves EQU—and CertOpt is correct.

Query 1:

```
SELECT DISTINCT COURSE.COURSE_ID, COURSE.TITLE,
      TEACHES.ID
FROM COURSE NATURAL JOIN TEACHES
WHERE COURSE.COURSE_ID = TEACHES.COURSE_ID
  AND TEACHES.YEAR = 2010
```

Query 2 (only the predicate differs):

```
SELECT DISTINCT COURSE.COURSE_ID, COURSE.TITLE,
      TEACHES.ID
FROM COURSE NATURAL JOIN TEACHES
WHERE COURSE.COURSE_ID >= TEACHES.COURSE_ID
  AND TEACHES.YEAR = 2010
```

Analysis. NATURAL JOIN equi-joins on *all shared column names*; here the only shared column is COURSE_ID. In every result row, `COURSE.COURSE_ID = TEACHES.COURSE_ID` holds by construction. Therefore = and >= in the WHERE clause are both tautologies over the join output, and the two queries are semantically identical.

VeriEQL (NEQ at k=1) produces a counterexample where COURSE_ID is 0 in COURSE but −1 in TEACHES, violating the join invariant. We verified in SQLite: both queries return identical (empty) results. CertOpt proves UNSAT up to k=3 (complete).

This case illustrates a broader observation: bounded verifiers that do not fully model implicit join predicates (e.g., NATURAL JOIN, JOIN USING) may produce spurious counterexamples.

G.8 Example 8: Scalar Subquery Cardinality Bug

Source: VeriEQL LeetCode benchmark [19], Problem 1083.

CertOpt correctly refutes a pair that VeriEQL proves equivalent up to k=32.

Query 1 (join-based):

```
SELECT DISTINCT S.BUYER_ID
FROM PRODUCT P
  INNER JOIN SALES S ON S.PRODUCT_ID = P.PRODUCT_ID
WHERE P.PRODUCT_NAME = 'S8'
  AND S.BUYER_ID NOT IN (
    SELECT S2.BUYER_ID FROM PRODUCT P2
      INNER JOIN SALES S2
        ON S2.PRODUCT_ID = P2.PRODUCT_ID
    WHERE P2.PRODUCT_NAME = 'iPhone')
```

Query 2 (scalar subquery):

```
SELECT DISTINCT BUYER_ID FROM SALES
WHERE PRODUCT_ID =
  (SELECT PRODUCT_ID FROM PRODUCT
   WHERE PRODUCT_NAME = 'S8')
  AND BUYER_ID NOT IN (
    SELECT DISTINCT BUYER_ID FROM SALES
    WHERE PRODUCT_ID =
      (SELECT PRODUCT_ID FROM PRODUCT
       WHERE PRODUCT_NAME = 'iPhone'))
```

Analysis. The schema has a primary key on PRODUCT_ID but *not* on PRODUCT_NAME. CertOpt synthesizes a witness with two products sharing the name 'S8' (distinct PRODUCT_ID values −2 and −1). Query 1's join correctly handles multiple matches, while Query 2's scalar subquery (SELECT PRODUCT_ID . . .) returns two rows—a runtime cardinality violation in SQL. We validated the disagreement in both DuckDB and SQLite. VeriEQL does not model scalar-subquery cardinality semantics, leading to a false equivalence proof.

G.9 Example 9: Self-Referencing FK with NULL Column

Source: VeriEQL LeetCode benchmark [19], Problem 1795.

Query 1:

```
SELECT NAME FROM CUSTOMER
WHERE REFEREE_ID <> 2 OR REFEREE_ID IS NULL
```

Query 2:

```
SELECT C1.NAME FROM CUSTOMER C1
  LEFT JOIN CUSTOMER C2 ON C1.REFEREE_ID = C2.ID
WHERE C2.NAME IS NULL OR C2.ID <> 2
```

Analysis. The table has a self-referencing FK (REFEREE_ID → ID) and a constraint REFEREE_ID ≠ ID. CertOpt's witness: two customers (ID=1, REFEREE_ID=2, NAME=NULL) and (ID=2, REFEREE_ID=1, NAME=NULL), satisfying all constraints. Query 1 returns one row (REFEREE_ID = 1 ≠ 2), while Query 2 returns two rows (both referees have NULL names, triggering C2.NAME IS NULL). VeriEQL proves EQU up to k=32 but misses the interaction between the LEFT JOIN projection and the NULL-aware WHERE clause.

G.10 Example 10: LLM Rewrite Rejection (TPC-DS)

Source: SQLStorm TPC-DS benchmark [35], LLM-generated rewrite (pair 18597).

Query 1:

```
SELECT c.c_customer_id,
       SUM(ss.ss_sales_price) AS total_sales
FROM customer AS c
JOIN store_sales AS ss
  ON c.c_customer_sk = ss.ss_customer_sk
GROUP BY c.c_customer_id
ORDER BY total_sales DESC
LIMIT 10;
```

Query 2:

```
WITH customer_totals AS (
    SELECT ss_customer_sk,
           SUM(ss_sales_price) AS total_sales
    FROM store_sales
    GROUP BY ss_customer_sk
)
SELECT c.c_customer_id, ct.total_sales
FROM customer AS c
JOIN customer_totals AS ct
  ON c.c_customer_sk = ct.ss_customer_sk
ORDER BY ct.total_sales DESC
LIMIT 10;
```

Result: NEQ in 1297 ms, witness validated.

Analysis. The LLM factored the aggregation into a CTE but changed the GROUP BY semantics. When a customer has multiple c_customer_sk values mapping to the same c_customer_id, the original groups by c_customer_id (merging them), while the CTE groups by ss_customer_sk (keeping them separate). CERTOPT's witness contains two store_sales rows with the same ss_customer_sk but different sales prices, demonstrating the multiplicity divergence.

G.11 Example 11: LLM Rewrite Rejection (StackOverflow)

Source: SQLStorm StackOverflow benchmark [35], LLM-generated rewrite (pair 12223).

Query 1:

```
SELECT pt.Name AS PostType,
       COUNT(p.Id) AS TotalPosts,
       AVG(p.Score) AS AvgScore,
       SUM(p.ViewCount) AS TotalViews
FROM Posts p
JOIN PostTypes pt ON p.PostTypeId = pt.Id
GROUP BY pt.Name
ORDER BY TotalPosts DESC;
```

Query 2:

```
WITH PostAgg AS (
    SELECT PostTypeId,
           COUNT(Id) AS TotalPosts,
           AVG(Score) AS AvgScore,
           SUM(ViewCount) AS TotalViews
    FROM Posts
    GROUP BY PostTypeId
)
SELECT pt.Name AS PostType,
       pa.TotalPosts, pa.AvgScore, pa.TotalViews
FROM PostAgg pa
```

```
JOIN PostTypes pt ON pa.PostTypeId = pt.Id
ORDER BY pa.TotalPosts DESC;
```

Result: NEQ in 1446 ms, witness validated.
Analysis. The LLM moved aggregation before the JOIN. When a `PostTypeId` in `Posts` has no matching row in `PostTypes`, the original query drops those posts (`INNER JOIN` filters before `GROUP BY`). The CTE computes aggregates over all posts including unmatched ones, then the JOIN filters—but the aggregates already incorporated the unmatched rows. CERTOPT's witness has a `Posts` row with `PostTypeId = -2` that does not exist in `PostTypes`, exposing the semantic difference.

H WINDOW FUNCTION ENCODING

Window functions are encoded exactly for ranking functions and partition-level aggregates, via a pre-computation pass that runs before the main expression evaluator.

H.1 Partition Identification

For a window function with `PARTITION BY` $p_1, \ldots, p_m$, two combos c_i and c_j belong to the same partition iff all partition keys agree under SQL equality (both-null-or-both-equal):

$$\text{same\_part}(c_i, c_j) = \bigwedge_{l=1}^{m} \Big[\big(p_l(c_i).null \wedge p_l(c_j).null\big) \vee \big(\neg p_l(c_i).null \wedge \neg p_l(c_j).null \wedge p_l(c_i).val = p_l(c_j).val\big) \Big]$$

H.2 Ranking Functions

ROW_NUMBER(). For each surviving combo c_i, compute the rank within its partition as 1+ the count of partition peers that are *strictly better* under the ORDER BY key. Ties are broken existentially: Z3 selects one of the possible ROW_NUMBER assignments consistent with the ORDER BY specification.

$$\text{row\_number}(c_i) = 1 + \sum_{j \neq i} \mathit{If}\big(\text{survives}(c_j) \wedge \text{same\_part}(c_i, c_j) \wedge \text{better}(c_j, c_i),\ 1,\ 0\big)$$

RANK(). Same as ROW_NUMBER but ties receive the same rank:

$$\text{rank}(c_i) = 1 + \sum_{j \neq i} \mathit{If}\big(\text{survives}(c_j) \wedge \text{same\_part}(c_i, c_j) \wedge \text{better}(c_j, c_i),\ 1,\ 0\big)$$

The difference from ROW_NUMBER is in the definition of "strictly better": RANK compares only the ORDER BY key, so ties (equal keys) yield the same rank.

DENSE_RANK(). 1+ the count of *distinct* better key-tuples in the partition. Uses first-occurrence deduplication: a key-tuple at combo c_j is counted only if no earlier combo $c_l < c_j$ in the partition has the same key-tuple:

$$\text{dense\_rank}(c_i) = 1 + \sum_{j} \mathit{If}\big(\text{first\_occ}(c_j) \wedge \text{better\_key}(c_j, c_i),\ 1,\ 0\big)$$

H.3 Partition Aggregates

For aggregate window functions (e.g., `SUM OVER (PARTITION BY ...)`), the encoder reuses the standard `_compute_aggregate` routine with the partition membership predicate replacing the GROUP BY same-group matrix. This avoids duplicating the aggregate encoding logic.

I COST MODEL DETAILS

CERTOPT supports three cost models, selectable via configuration.

I.1 Syntactic Cost Model

A lightweight heuristic that scores query structure without executing any database operations:

Feature	Weight
Each INNER JOIN	+10
Each CROSS JOIN	+100
Each subquery	+20
DISTINCT present	+5
Each GROUP BY column	+3
ORDER BY present	+5
LIMIT present	−2
Each table in FROM	+1
Late WHERE filter*	+2

*A WHERE conjunct referencing a joined table that could benefit from pushdown.

The late-WHERE-filter penalty encourages predicate pushdown rewrites (R1) to register as cost improvements even when the overall query structure is otherwise unchanged.

I.2 EXPLAIN-Based Cost Models

SQLite EXPLAIN.. Parses the output of `EXPLAIN QUERY PLAN` and assigns: SCAN = 100, SEARCH = 10, TEMP B-TREE = 20, COMPOUND = 15.

DuckDB EXPLAIN.. Parses DuckDB's `EXPLAIN` plan tree, assigning operator-level costs: SEQ_SCAN = 100, HASH_JOIN = 15, FILTER = 5, plus log-scaled row estimates as a tiebreaker.

J CONFIGURATION AND ABLATION PRESETS

The `OptimizerConfig` dataclass centralizes all pipeline feature flags, supporting systematic ablation studies. Table 12 lists the named presets.

Table 12: Ablation presets for controlled experiments. Each preset disables one component to measure its contribution.

Preset	What it disables
`no_llm`	LLM rewrite generation
`no_rules`	Rule-based rewrite generation
`no_preprocessing`	Predicate promotion + table elimination
`no_family_pruning`	Counterexample-guided family pruning
`no_verification`	Z3 witness synthesis (accept all structurally valid)
`rules_only`	LLM rewrites (rules only)
`llm_only`	Rule rewrites (LLM only)

Key configuration parameters:

- **Bounded scope Σ:** k (rows per table, default 3), integer domain bounds (widened by query literals), solver timeout (default 10 s).
- **Combo limits:** 64 (unpreprocessed), 256 (preprocessed + complex), 512 (preprocessed + non-complex). Compositional local regions use 65,536.
- **CEGAR iterations:** up to 3 refinement rounds when spurious witnesses are detected.
- **Adaptive k:** compositional regions with ≥ 5 tables use $k = 1$.

K INTEGRITY CONSTRAINT ENCODING

CertOpt encodes four classes of integrity constraints as Z3 assertions added to the solver before the difference predicate.

Primary Key / UNIQUE.. For each PK or UNIQUE constraint on columns $(c_1, \ldots, c_m)$ of table T with k symbolic rows, assert pairwise distinctness for all $\binom{k}{2}$ row pairs (r_i, r_j):

$$\bigvee_{l=1}^{m} \left(v_{c_l,i} \neq v_{c_l,j}\right) \;\vee\; \left(n_{c_l,i} \oplus n_{c_l,j}\right)$$

where $v_{c,r}$ is the value and $n_{c,r}$ is the null flag of column c in row r.

Foreign Key. For each FK constraint $T_1.c_{\text{fk}} \rightarrow T_2.c_{\text{pk}}$, assert that every non-null FK value matches some PK value:

$$\forall r_i \in T_1 :\; n_{\text{fk},i} \;\vee \bigvee_{r_j \in T_2} \left(\neg n_{\text{pk},j} \wedge v_{\text{fk},i} = v_{\text{pk},j}\right)$$

Reverse FK Guard. When a foreign key's source column is itself a primary key of its table, the FK constraint represents a *parent→child* back-reference rather than a child→parent dependency. In a bounded-k encoding, forcing every parent PK row to match some child FK row would make nullable FK columns effectively `NOT NULL`, producing false equivalence verdicts (the solver cannot explore instances where a parent row has no children). Therefore, when the source column of an FK is a primary key, the FK constraint is *suppressed*—only the child→parent direction is enforced. This is the safe direction: suppression enlarges the solution space (false Sat possible, false Unsat impossible).

NOT NULL.. For each NOT NULL column c in table T: $\forall r_i \in T : \neg n_{c,i}$.

L CEGAR REFINEMENT LOOP

When witness synthesis returns SAT but the witness is *spurious*—i.e., both queries agree when executed on the materialized witness database—CertOpt enters a CEGAR (Counterexample-Guided Abstraction Refinement) loop:

(1) **Detect spurious witness:** Materialize the SAT model as a DuckDB (or SQLite) database. Execute both original and rewrite SQL. Compare results as multisets. If they agree, the witness is spurious (caused by conservative approximation of an unmodeled construct).
(2) **Add blocking constraint:** Extract the concrete cell values from the spurious model and assert their negation as a blocking clause, preventing the solver from producing the same assignment again.
(3) **Re-invoke solver:** Re-check satisfiability with the blocking constraint added. If SAT again, validate the new witness. If UNSAT, the queries are equivalent (the spurious witness was the only counterexample).
(4) **Iterate:** Repeat up to 3 times. If all witnesses in 3 rounds are spurious, downgrade the result to UNKNOWN.

This loop reduces false rejection rates without compromising soundness: UNSAT after blocking still implies equivalence (the enlarged constraint space is a subset of the original), and confirmed SAT witnesses are genuine counterexamples.

M BOUNDED-k COMPLETENESS GUARDS

Certain SQL patterns require a minimum number of rows per table to distinguish semantically different queries. CertOpt implements three completeness guards that detect when the chosen bound k may be insufficient.

Guard 1: HAVING COUNT threshold. If either query contains `HAVING COUNT(*) >= n` (or `> n−1`), then k must be at least n for the verifier to produce a group with enough members. If $k < n$, synthesis may return a vacuous UNSAT (no group can satisfy the HAVING predicate at this bound).

Guard 2: Derived-table GROUP BY asymmetry. If Q_1 contains a derived table `SELECT ... FROM T GROUP BY ...` but Q_2 does not (or vice versa), the aggregation collapses multiplicities differently. At low k, the collapsed result may trivially match. The guard flags this asymmetry.

Guard 3: Self-join asymmetry. If Q_1 uses T with c_1 aliases and Q_2 uses T with $c_2 \neq c_1$ aliases, and $\max(c_1, c_2) \geq k$, the bounded domain may not provide enough distinct values for the extra self-join to produce a distinguishing witness.

When a guard fires, the engine *immediately returns* Unknown without invoking the solver. This prevents vacuous UNSAT verdicts from being misinterpreted as equivalence proofs. The guard check runs once against the maximum k in the at-most-k schedule before the loop begins; if it fires, no solver call is made at any cardinality.

N CERTIFICATE REPLAY VALIDATION

To validate the "replayable proof artifact" claim (§7.1), we ran an end-to-end certificate generation and replay experiment on the VeriEQL LeetCode benchmark.

Protocol. From 500 EQU pairs in the LeetCode suite, we selected those for which CertOpt could re-prove equivalence at the originally recorded bound k (up to k=3). For each pair, we: (1) reran witness synthesis to confirm Unsat, (2) built an equivalence certificate recording both normalized IRs, the scope Σ, and cryptographic hashes of the IRs, rendered SQL, and catalog schema, and (3) serialized the certificate and a catalog snapshot to disk as `certificate.json`. All 500 pairs produced valid certificates.

In a separate *replay* phase, we loaded each certificate from disk (without access to the original verification context), reconstructed both IRs from the serialized JSON, verified all cryptographic hashes (IR, SQL, catalog), re-ran the equivalence witness synthesis from scratch, and compared the solver outcome against the recorded status.

Results. All 500 certificates replayed successfully with matching solver outcomes and hash integrity (100% pass rate). The mean replay time was 110 ms (median 42 ms, p95 424 ms), confirming that certificate replay is practical for independent re-verification.

Table 13: Certificate replay validation (LeetCode benchmark, $k \leq 3$).

Metric	Value
Input EQU pairs	500
Certificates generated	500
Replay passed	500 / 500 (100%)
Replay failed	0
Mean replay time	110 ms
Median replay time	42 ms
p95 replay time	424 ms
Max replay time	578 ms

O FAMILY PRUNING ABLATION

We evaluate the impact of counterexample-guided family pruning (§4.3) on the BIRD benchmark [25]. The dataset contains 500 queries with 3,071 LLM-generated candidates (mean 6.1 per query). Candidates are grouped by *source family*: original LLM outputs, DISTINCT variants, aggregate function mutations, and others. For each query, candidate [0] serves as the specification and all remaining candidates are verified against it using monolithic witness synthesis at k=2 with a 5,s timeout.

Table 14 compares two conditions: (i) every candidate verified individually (*no pruning*), and (ii) when a candidate returns SAT, all remaining candidates in the same source family are pruned without calling the solver (*with pruning*).

Table 14: Family pruning ablation on BIRD (500 queries, 3,071 candidates). Pruning reduces solver calls by 33.1% and solver time by 25.6%.

Metric	No Pruning	With Pruning	Savings
Solver calls	3,071	2,056	1,015 (33.1%)
Candidates pruned	0	1,015	—
SAT (rejected)	1,464	678	—
UNSAT (equivalent)	547	467	—
Unknown / timeout	1,060	911	—
Solver time	788.9 s	587.1 s	201.8 s (25.6%)

The SAT and UNSAT counts differ between the two conditions because pruning occurs *before* solver invocation: the 1,015 pruned candidates include 786 that would have been individually rejected (SAT), 80 that would have been confirmed equivalent (UNSAT), and 149 that would have returned unknown or timed out. The 80 missed UNSAT results represent potential optimization opportunities forgone by pruning, not soundness violations—every candidate that survives pruning is still individually verified.

The pruned candidates come predominantly from DISTINCT variants (1,464 total members), confirming that mutation families share structural defects: once the solver rejects one DISTINCT variant, the remaining variants in the same query are pruned without individual verification.

Time savings. The reported 201.8 s reduction is the measured wall-clock difference between the two runs on identical hardware. With pruning the average cost per solver call *increases* from 257 ms to 286 ms: pruning disproportionately removes fast SAT rejections (77.4% of pruned candidates), so the residual call set has a higher proportion of expensive unknown/timeout results (44.3% vs. 34.5%). The overall time saving therefore understates the true benefit of avoided work: the 33.1% reduction in solver calls yields a 25.6% time reduction despite the remaining calls being individually more expensive.

P SPURIOUS WITNESS ROOT CAUSE ANALYSIS

Across all three VeriEQL suites, 4,597 pairs receive an UNKNOWN verdict from CertOpt: the Z3 solver finds a satisfying assignment (SAT) but executing both queries on the materialized witness database produces identical results, revealing the witness as spurious. We performed an instance-by-instance root cause analysis, classifying each spurious witness by the *encoding precision gap* it exploits. Table 15 summarizes the results.

Table 15: Root cause analysis of 4,597 spurious witnesses by encoding precision gap.

Encoding Gap	Count	%	Mechanism
Numeric type coercion	1,662	36.2	Z3 exact rational vs. SQL float
Row-choice nondeterminism	1,319	28.7	Scalar subquery / LIMIT tie-break
Fresh-variable fallback	1,278	27.8	Unmodeled func → unconstrained var
Bounded-k incompleteness	188	4.1	HAVING threshold exceeds k=3
DT result truncation	150	3.3	Intermediate result capped at k rows

Numeric type coercion (36.2%). The Z3 encoding uses RealSort with integrality constraints for integer columns. Division produces exact rationals (e.g., $7/2 = 3.5$) rather than SQL's typed arithmetic ($7/2 = 3$ for integers). `CAST` is modeled as identity (except for `BOOLEAN`). When `ROUND(AVG(...), n)` appears, Z3 rounds the exact rational average, which may differ from rounding the float computed by SQL—even though both results agree when executed on the same concrete database.

Row-choice nondeterminism (28.7%). Scalar subqueries are encoded as "pick the first surviving row" without enforcing singleton semantics. `ORDER BY + LIMIT` uses existential tie-break variables. When Q1 uses a correlated scalar subquery and Q2 rewrites it as a `JOIN`, the two go through different encoding paths; Z3 can construct models where the row-choice diverges, producing a spurious difference.

Fresh-variable fallback (27.8%). Functions not in the modeled fragment (e.g., `DATE()`, `SUBSTR()`, window functions) are assigned unconstrained Z3 variables. The solver can set these to arbitrary values that create a difference between Q1 and Q2, but real execution always agrees. Witness values containing `__fresh_lo__` or `__fresh_hi__` markers are direct evidence of this gap.

Bounded-k incompleteness (4.1%). Queries with `HAVING COUNT(*) >= 5` require at least 5 rows per group, but the default k=3 bound

cannot represent such databases. The solver finds SAT on the truncated encoding.

DT result truncation (3.3%). Derived-table and CTE results are capped at k rows. Even with correct base-table bounds, truncating intermediate results can change outer query semantics.

All three dominant gaps (92.7%) are inherent to bounded SMT-based verification over finite arithmetic sorts.

Precision–completeness tradeoff. Paradoxically, VeriEQL proves equivalence on 2,616 of these pairs where CertOpt returns UNKNOWN. The reason is that the two tools make opposite encoding choices. VeriEQL uses IntSort for all values, so division is automatically truncating—matching SQL integer semantics. It also treats `ROUND`, `CAST`, and type conversions as uninterpreted functions: $f(x) = f(x)$ holds trivially, so structurally identical expressions are guaranteed equal regardless of what f computes. CertOpt uses RealSort and models `ROUND` exactly, yielding a *more precise* encoding that can distinguish cases VeriEQL cannot—but this precision also lets the solver exploit gaps between Z3 rational arithmetic and SQL float arithmetic, producing spurious witnesses.

This is a principled tradeoff: CertOpt's precision is what enables the 241 confirmed VeriEQL false equivalence proofs (§8.2), 153 of which are caused by exactly the integer-vs-rational division gap that VeriEQL's coarser encoding cannot detect. The cost is a higher UNKNOWN rate on pairs where the encoding is precise enough to *see* a difference that does not manifest in real execution. CertOpt mitigates this via witness validation: every SAT result is executed on the materialized database, and spurious witnesses are downgraded to UNKNOWN rather than reported as non-equivalent.